

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту

на тему:

**ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА
МОНИТОРИНГА И ОБЕСПЕЧЕНИЯ
БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ**

БГУИР ДП 1-40 03 01 001 ПЗ

Студент

Д. В. Быльков

Руководитель

Д. С. Кисляк

Консультанты:

от кафедры ИИТ

В. В. Захаров

по экономической части

Т. Л. Слюсарь

Нормоконтролёр

В. В. Захаров

Рецензент

Минск 2026

Решением рабочей комиссии
допущен(а) к защите дипломного проекта

Председатель рабочей комиссии

Д. В. Шункевич

(Подпись)

(Инициалы и Фамилия)

2026 г.

Учреждение образования
Белорусский государственный университет
информатики и радиоэлектроники
Кафедра интеллектуальных информационных технологий

УТВЕРЖДАЮ

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

_____ 2026 г.

ЗАДАНИЕ
на дипломный проект

Обучающемуся _____ Былькову Даниилу Владимировичу

(фамилия, собственное имя, отчество (если таковое имеется))

Курс 4 _____ Учебная группа 221703

Специальность 1-40 03 01 Искусственный интеллект

Тема дипломного проекта

Интеллектуальная система мониторинга и обеспечения безопасности путешествия

(наименование темы)

Утверждена руководителем УВО БГУИР №632-с от 25 марта 2024 г.

Исходные данные к дипломному проекту: Сервер с предустановленной операционной системой Ubuntu Server; Языки разработки Java; Python.

Перечень подлежащих разработке вопросов или краткое содержание расчетно-пояснительной записки: _____

Введение

1 Анализ ...

2 Проектирование ...

3 Разработка ...

4 Технико-экономическое обоснование ...

Заключение

Перечень графического материала (с точным указанием обязательных чертежей и графиков): _____

Схема алгоритма согласованного управления оборудованием — формат А4, лист 1.

— формат А4, лист 1.

— формат А4, лист 1.

— формат А4, лист 1.

— формат А4, лист 1.

— формат А4, лист 1.

Консультанты по дипломному проекту (с указанием разделов, по которым они консультируют): _____

Раздел 1–3 — консультант от кафедры И.О. Фамилия

Раздел 4 — консультант по экономической части И.О. Фамилия

Примерный календарный график выполнения дипломного проекта

№ п/п	Наименование этапа дипломного проекта	Объем этапа, %	Срок выполнения этапа
1	Обзор литературных источников по теме, изучение проблемной области	10	25.03 – 21.04
3	Определение требований к реализации	10	30.03 – 20.04
4	Проектирование модели подсистемы	25	10.04 – 30.04
5	Разработка подсистемы	30	21.04 – 20.05
6	Экономическое обоснование принятого решения, определение экономической эффективности внедрения полученных результатов	10	01.05 – 20.05
7	Оформление расчетно-пояснительной записки	10	30.03 – 29.05
8	Оформление графического материала	5	01.05 – 29.05

Дата выдачи задания 25 марта 2026 г.

Срок сдачи студентом законченного дипломного проекта 30 мая 2026 г.

Руководитель дипломного проекта _____
(подпись)

И. О. Фамилия
(инициалы, фамилия)

Подпись обучающегося _____

Дата _____ 2026 г.

РЕФЕРАТ

НАИМЕНОВАНИЕ ТЕМЫ ДИПЛОМНОГО ПРОЕКТА: дипломный проект/ И. О. Фамилия. – Минск : БГУИР, 2026, – п.з. – 60 с., чертежей (плакатов) – 6 л. формата А1.

Целью дипломного проекта является

Предметом исследования является ...

Данный программный модуль позволяет

В первом разделе пояснительной записки проведен анализ

Во втором разделе

В третьем разделе

В четвертом разделе приведено технико-экономическое обоснование разрабатываемого программного продукта.

Результатом дипломного проектирования является программа ...

СОДЕРЖАНИЕ

Список принятых сокращений	7
Введение	8
1 Анализ подходов к разработке интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников . . .	12
1.1 Анализ предметной области	12
1.1.1 Результаты анализа предметной области	14
1.2 Анализ подходов к решению задачи	14
1.2.1 Обоснование выбора интеллектуального анализа	15
1.3 Анализ аналогов системы	16
1.3.1 International SOS	16
1.3.2 Safeture	17
1.3.3 Crisis24	18
1.3.4 Сравнение аналогов	19
1.3.5 Выводы по результатам исследования существующих решений	19
1.4 Анализ источников данных	20
1.4.1 Характеристика информационного фундамента системы	20
1.5 Анализ архитектурных подходов	21
1.5.1 Оценка эффективности выбранной архитектурной модели	21
1.6 Выбор моделей и методов машинного обучения	21
1.6.1 Обоснование состава нейросетевых моделей	22
1.7 Анализ технологического стека	22
1.7.1 Резюме по выбору технологических инструментов	23
1.7.2 Выводы по результатам анализа	23
2 Проектирование системы мониторинга рисков и обеспечения безопасности путешественников	24
2.1 Функциональные требования и сценарии использования	24
2.2 Архитектура системы и стратегия развертывания	25
2.2.1 Модуль сбора данных (Fetcher)	26
2.2.2 Модуль интеллектуального анализа событий (Analyzer)	27
2.2.3 Серверная часть приложения (Core Service)	28
2.3 Взаимодействие компонентов системы	29
2.4 Проектирование структуры базы данных	29
2.4.1 Промежуточное хранилище (Staging Layer)	30
2.4.2 Основное хранилище (Production Layer)	30
2.5 Проектирование программных интерфейсов	31
2.6 Выводы по проектированию	31

3	Разработка интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников	32
3.1	Разработка модуля сбора и первичной обработки данных . .	32
3.2	Программная реализация интеллектуального анализа на Python	34
3.3	Реализация серверной логики и ГИС-компонентов на Java .	35
3.4	Реализация системы многоканальных уведомлений	37
3.5	Разработка клиентского интерфейса и ГИС-компонентов . .	38
3.6	Выводы по реализации	39
4	Технико-экономическое обоснование разработки и использования интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников по индивидуальному заказу . . .	44
4.1	Характеристика разработанного по индивидуальному заказу программного средства интеллектуального мониторинга рисков и обеспечения безопасности путешественников	44
4.2	Расчет затрат на разработку и цены интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников, созданной по индивидуальному заказу	45
4.2.1	Расчет затрат на основную заработную плату разработчикам	45
4.2.2	Расчет затрат на дополнительную заработную плату разработчикам	46
4.2.3	Расчет отчислений на социальные нужды	47
4.2.4	Расчет затрат на прочие расходы	47
4.2.5	Расчет суммы затрат на разработку и цены программного средства	48
4.2.6	Результаты расчета затрат на разработку и цены программного средства	49
4.3	Расчет результата от разработки и использования интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников, созданной по индивидуальному заказу	49
4.4	Расчет показателей экономической эффективности разработки и использования интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников для организации-заказчика	51
4.5	Выводы по результатам расчета	53
	Заключение	54

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

AI— Искусственный интеллект

API— Интерфейс прикладного программирования

GDELT— Глобальная база данных событий, языка и тональности

GEOJSON— Формат обмена пространственными данными на основе JSON

JTS— Библиотека для манипулирования геоданными для языка программирования Java

LLM— Большая языковая модель

ML— Машинное обучение

NER— Распознавание именованных сущностей

NLP— Обработка естественного языка

OSM— OpenStreetMap (Открытая геопространственная база данных)

REST— Передача состояния представления

TRM— Управление рисками в путешествиях

UML— Унифицированный язык моделирования

VRAM— Видеопамять

ВВЕДЕНИЕ

Современный этап развития мировых социально-экономических отношений характеризуется беспрецедентным уровнем глобализации и устойчивым ростом интенсивности международных перемещений. Несмотря на периодические колебания, вызванные глобальными кризисами, общая тенденция к расширению географии деловых поездок и индивидуального туризма сохраняет свою актуальность. В этих условиях обеспечение безопасности путешественников становится одной из приоритетных задач как для корпоративного сектора, так и для специализированных туристических компаний. Развитие концепции «должной осмотрительности» (Duty of Care), закрепленной в международных стандартах, таких как ISO 31030, возлагает на организации юридическую и моральную ответственность за жизнь и здоровье своих клиентов и сотрудников во время их нахождения за рубежом.

Ландшафт глобальных угроз в последние годы претерпел существенные изменения, став значительно более динамичным, многофакторным и труднопредсказуемым. Путешественники ежедневно сталкиваются с широким спектром рисков: от внезапных геополитических конфликтов, массовых протестов и террористических актов до стремительно развивающихся природных катаклизмов, вызванных климатическими изменениями. Эпидемиологические угрозы, сбои в работе критической транспортной инфраструктуры и возрастающий уровень локальной криминогенной напряженности требуют непрерывного контроля. В такой нестабильной среде традиционные методы оценки рисков, опирающиеся на статические страновые отчеты и периодические сводки, демонстрируют свою полную несостоятельность.

Анализ достижений в области управления рисками в путешествиях показывает, что существующие на рынке решения исторически развивались по двум основным направлениям. Первое направление базируется на экспертно-аналитическом подходе, при котором штат специалистов вручную осуществляет мониторинг новостных лент, верификацию информации и рассылку предупреждений. Данный метод обеспечивает высокую точность и отсутствие ложных срабатываний, однако характеризуется критически низкой скоростью реакции (задержки могут достигать нескольких часов) и колоссальными операционными издержками, что делает такие системы недоступными для малого бизнеса и индивидуальных туристов. Второе направление использует алгоритмический сбор данных на основе жестко заданных ключевых слов. Несмотря на высокую скорость обработки информации, подобные агрегаторы генерируют неприемлемый уровень информационного шума, поскольку алгоритмы не способны улавливать контекст, тональность сообщения и отличать реальную угрозу от исторической справки или метафоры в тексте новостной статьи.

Настоящим прорывом в решении проблемы глобального мониторин-

га стало стремительное развитие технологий искусственного интеллекта, в частности, методов обработки естественного языка (Natural Language Processing, NLP) и больших языковых моделей (Large Language Models, LLM). Появление архитектур на базе трансформеров позволило перейти от простого синтаксического поиска к глубокому семантическому анализу неструктурированных текстовых данных. Использование методов Zero-shot классификации дает возможность сопоставлять поступающие новостные сообщения с обширной таксономией угроз без необходимости постоянного переобучения моделей на новых наборах данных. Интеграция алгоритмов продвинутого геопарсинга позволяет с высокой точностью извлекать из текста пространственные координаты описываемых событий, формируя зоны потенциального поражения. Совокупность этих инновационных подходов открыла возможность создания систем мониторинга, способных обрабатывать огромные массивы информации из систем открытой разведки (OSINT), таких как проект GDELT, в режиме реального времени, извлекая только релевантные данные и сводя к минимуму участие человека в процессе первичной фильтрации.

Целью дипломного проектирования является разработка интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников по индивидуальному заказу туристической компании ООО «ТревелСейф».

Реализация данной цели направлена на автоматизацию процесса сбора, семантического анализа и оперативного оповещения об угрозах для обеспечения принципа «должной осмотрительности» и надежной защиты клиентов компании во время их поездок.

В основу проектирования, научного исследования и поиска технического решения положены следующие принципы:

- использование событийно-ориентированной микросервисной архитектуры для обеспечения высокой отказоустойчивости и сверхвысокой скорости обработки независимых потоков данных;
- применение передовых моделей машинного обучения для глубокого семантического анализа открытых источников без необходимости ручной модерации входящего контента;
- внедрение алгоритмов точного геопарсинга для автоматического извлечения пространственных координат инцидентов из неструктурированных текстовых массивов;
- интеграция механизмов геофенсинга для непрерывного сопоставления текущего местоположения пользователя или его планируемого маршрута с динамически изменяющимися зонами риска;
- применение локальных больших языковых моделей для автоматического реферирования новостных сводок и генерации лаконичных уведомлений;

- изоляция промежуточных сырых данных от очищенной аналитической информации на уровне структуры базы данных для обеспечения чистоты анализа;

- ориентация на конечного пользователя посредством создания интуитивно понятного интерфейса с интерактивной глобальной картой и гибкой системой настройки профиля угроз.

Для достижения поставленной цели в рамках дипломного проектирования необходимо решить следующие задачи:

- провести комплексный анализ предметной области и существующих на мировом рынке систем управления рисками в путешествиях;

- выявить недостатки текущих аналогов и обосновать необходимость применения интеллектуального подхода на базе нейронных сетей;

- выполнить сравнительный анализ источников данных о глобальных инцидентах и выбрать наиболее оптимальные агрегаторы;

- обосновать выбор архитектурного паттерна, технологического стека и конкретных моделей машинного обучения для реализации вычислительного ядра системы;

- спроектировать микросервисную архитектуру программного комплекса и разработать схемы взаимодействия его внутренних компонентов;

- разработать структуру реляционной базы данных с поддержкой пространственных запросов для хранения информации об инцидентах и пользователях;

- рассчитать экономические показатели целесообразности инвестиций в разработку и внедрение программного средства для организации заказчика.

Пояснительная записка к дипломному проекту структурно состоит из введения, трех основных разделов и заключения.

Краткое изложение содержания разделов пояснительной записки:

- в первом разделе проводится анализ подходов к разработке интеллектуальной системы мониторинга рисков, исследуются существующие на рынке аналогичные решения, подробно рассматриваются источники данных о глобальных событиях, а также обосновывается выбор технологий, фреймворков и моделей машинного обучения;

- во втором разделе приведено описание функциональных требований к системе, выполнено архитектурное проектирование ее микросервисных компонентов, а также представлено детальное описание структуры базы данных, программных интерфейсов и алгоритмов взаимодействия модулей сбора, анализа и оповещения;

- в третьем разделе осуществляется экономическое обоснование целесообразности инвестиций в разработку программного средства, включающее расчет затрат на заработную плату разработчиков, формирование отпускной цены системы, а также расчет показателей экономической эффективности,

таких как чистый дисконтированный доход и срок окупаемости;

– в заключении подводятся итоги проделанной работы, формулируются основные выводы по результатам проектирования и экономического обоснования, а также подтверждается успешное достижение поставленной цели дипломного проекта.

Дипломный проект выполнен самостоятельно, проверен в системе "Антиплагиат". Процент оригинальности соответствует норме, установленной кафедрой ИИТ. Цитирования обозначены ссылками на публикации, указанные в "Списке использованных источников".

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ

В данном разделе проводится глубокий и всесторонний анализ сферы управления рисками в путешествиях, исследуются существующие на рынке аналогичные решения и выявляются их недостатки. Также подробно рассматриваются источники данных о глобальных событиях, проводится сравнительный анализ архитектурных подходов и обосновывается выбор конкретных технологий, фреймворков и моделей машинного обучения, необходимых для реализации интеллектуальной системы мониторинга и оповещения.

1.1 Анализ предметной области

Системы управления рисками в путешествиях (Travel Risk Management, TRM) представляют собой комплексные программно-аппаратные и организационные решения, предназначенные для заблаговременного выявления, оценки, мониторинга и минимизации угроз, с которыми могут столкнуться сотрудники корпораций или частные путешественники. Развитие глобализации и увеличение объемов международных перемещений привели к тому, что обеспечение безопасности поездок стало одной из ключевых задач корпоративного управления.

Современная концепция TRM базируется на принципе «должной осмотрительности» (Duty of Care), согласно которой организация несет юридическую и моральную ответственность за безопасность своих сотрудников, находящихся в командировках. В 2021 году был опубликован международный стандарт ISO 31030 «Travel risk management — Guidance for organizations»[1], который систематизировал подходы к оценке рисков и установил высокие требования к системам мониторинга. Данный стандарт подчеркивает необходимость не только реактивного реагирования на инциденты, но и проактивного анализа потенциальных угроз еще на этапе планирования поездки.

Ландшафт глобальных угроз в середине 2020-х годов характеризуется высочайшей динамичностью и непредсказуемостью. В рамках исследования предметной области была разработана расширенная таксономия рисков: **Геополитические и социально-политические риски:**

– вооруженные конфликты, локальные боестолкновения и пограничные инциденты;

- государственные перевороты, резкая смена политических режимов и гражданское неповиновение;
- массовые протесты, забастовки работников транспортной отрасли и гражданские беспорядки;
- террористические акты, угрозы экстремистской деятельности и диверсии на объектах инфраструктуры;
- дипломатические инциденты, изменения в визовой политике и введение санкционных ограничений.

Природные и климатические риски:

- сейсмическая активность, включающая землетрясения различной магнитуды и угрозы возникновения цунами;
- метеорологические экстремумы, такие как ураганы, тайфуны, наводнения и аномальные температурные волны;
- лесные пожары, извержения вулканов и пылевые бури, ограничивающие видимость и работу авиации;
- долгосрочные климатические изменения, влияющие на устойчивость инфраструктуры конкретного региона.

Эпидемиологические и медицинские угрозы:

- вспышки инфекционных заболеваний, локальные очаги эпидемий и глобальные пандемии;
- низкий уровень развития местной системы здравоохранения и дефицит квалифицированного персонала;
- отсутствие доступа к специфическим медикаментам или качественной скорой помощи;
- специфические санитарно-гигиенические риски, связанные с качеством воды и пищевых продуктов.

Криминальные и кибернетические угрозы:

- уличная преступность, нападения с целью грабежа и мошенничество в отношении туристов;
- похищения людей с целью выкупа или политического давления;
- кибершпионаж через публичные незащищенные сети связи и фишинговые атаки;
- кража персональных данных, документов и физических носителей информации у путешественников.

Техногенные и инфраструктурные риски:

- аварии на объектах критической инфраструктуры, включая энергетические и водопроводные сети;
- масштабные сбои в работе транспортных узлов, аэропортов и железнодорожных вокзалов;
- энергетические кризисы, приводящие к веерным отключениям электричества и связи.

Эффективность любой системы мониторинга определяется «индексом

своевременности» (Time-to-Alert) — временным интервалом между возникновением инцидента и получением уведомления конечным пользователем. Для глобальных систем этот показатель должен стремиться к значениям менее 15 минут, что невозможно реализовать без глубокой автоматизации процессов сбора и анализа данных.

1.1.1 Результаты анализа предметной области

В ходе проведенного исследования было установлено, что сфера управления рисками в путешествиях трансформировалась из узкоспециализированной области безопасности в критически важный элемент корпоративного управления и личной безопасности. Современные стандарты, такие как ISO 31030, диктуют необходимость создания систем, способных обрабатывать огромные потоки информации в режиме реального времени. Выявленная таксономия рисков демонстрирует многогранность угроз, что требует применения междисциплинарного подхода при проектировании архитектуры интеллектуальной системы.

1.2 Анализ подходов к решению задачи

При разработке систем мониторинга безопасности путешественников выделяют три фундаментальных подхода к сбору и обработке данных, каждый из которых имеет свои преимущества и ограничения.

Первый подход — экспертно-аналитический. Он основан на работе штатных аналитиков, которые вручную отбирают новости из проверенных источников, проводят верификацию данных через собственные каналы и формируют аналитические отчеты. Преимущества данного подхода:

- высочайшая точность, глубина контекстуального анализа и достоверность предоставляемых сведений;
- практически полное отсутствие ложных срабатываний, вызванных метафоричностью или иронией в текстах;
- возможность составления долгосрочного прогноза развития ситуации на основе инсайдерской информации.

Недостатки:

- критически низкая скорость реакции на динамические инциденты из-за человеческого фактора;
- чрезвычайно высокая стоимость содержания круглосуточного штата высококвалифицированных аналитиков;
- ограниченность масштабирования, не позволяющая одновременно отслеживать тысячи локальных источников на разных языках.

Второй подход — алгоритмический сбор на основе ключевых слов и регулярных выражений. Системы настраиваются на автоматический поиск

определенных комбинаций слов в новостных лентах и социальных сетях. Преимущества:

- высокая скорость первичной обработки данных и возможность охвата большого количества источников;
- относительно низкая стоимость реализации и простота технической поддержки алгоритмов.

Недостатки:

- критически высокий уровень информационного шума и ложных срабатываний на нерелевантные новости;
- значительные трудности с автоматическим определением точной геолокации и границ зоны инцидента;
- неспособность алгоритмов понимать сложный контекст, тональность сообщения и выявлять дезинформацию.

Третий подход — интеллектуальный анализ с применением NLP (Natural Language Processing) и современных моделей машинного обучения. Данный подход предполагает использование глубоких нейронных сетей для семантического разбора текста, извлечения сущностей и автоматической многофакторной классификации. Преимущества:

- практически мгновенная обработка входящего потока информации в режиме реального времени;
- глобальный охват источников на множестве языков без необходимости пропорционального расширения штата;
- возможность автоматического геокодирования и динамической оценки уровня серьезности каждого инцидента.

Недостатки:

- высокая инженерная сложность разработки, обучения и интеграции нейросетевых моделей;
- значительная требовательность к вычислительным ресурсам, особенно при использовании больших языковых моделей;
- необходимость внедрения механизмов постоянного дообучения моделей на свежих данных для сохранения точности.

В результате проведенного анализа подходов к решению задачи, наиболее предпочтительным к реализации признан третий подход, дополненный гибридными механизмами автоматической проверки достоверности, что позволяет эффективно объединить скорость алгоритмов с качеством экспертной аналитики.

1.2.1 Обоснование выбора интеллектуального анализа

Сравнение различных методологий обработки информации показало, что традиционные методы не справляются с современными объемами данных и требованиями к скорости оповещения. Интеллектуальный подход на

базе NLP позволяет решить проблему масштабируемости и многоязычности, обеспечивая при этом приемлемый уровень точности за счет использования трансформерных архитектур. Таким образом, внедрение технологий машинного обучения является безальтернативным путем для создания конкурентоспособной системы мониторинга рисков.

1.3 Анализ аналогов системы

На мировом рынке представлено несколько крупных систем мониторинга рисков, ориентированных преимущественно на сегмент B2B (Business-to-Business), где заказчиками выступают крупные корпорации, страховые компании или глобальные туристические операторы.

1.3.1 International SOS

International SOS является признанным мировым лидером в области медицинского сопровождения и обеспечения безопасности. Их система мониторинга опирается на уникальную сеть из 27 центров помощи и тысячи штатных экспертов. На рисунке 1.1 представлен пример интерфейса мобильного приложения системы.

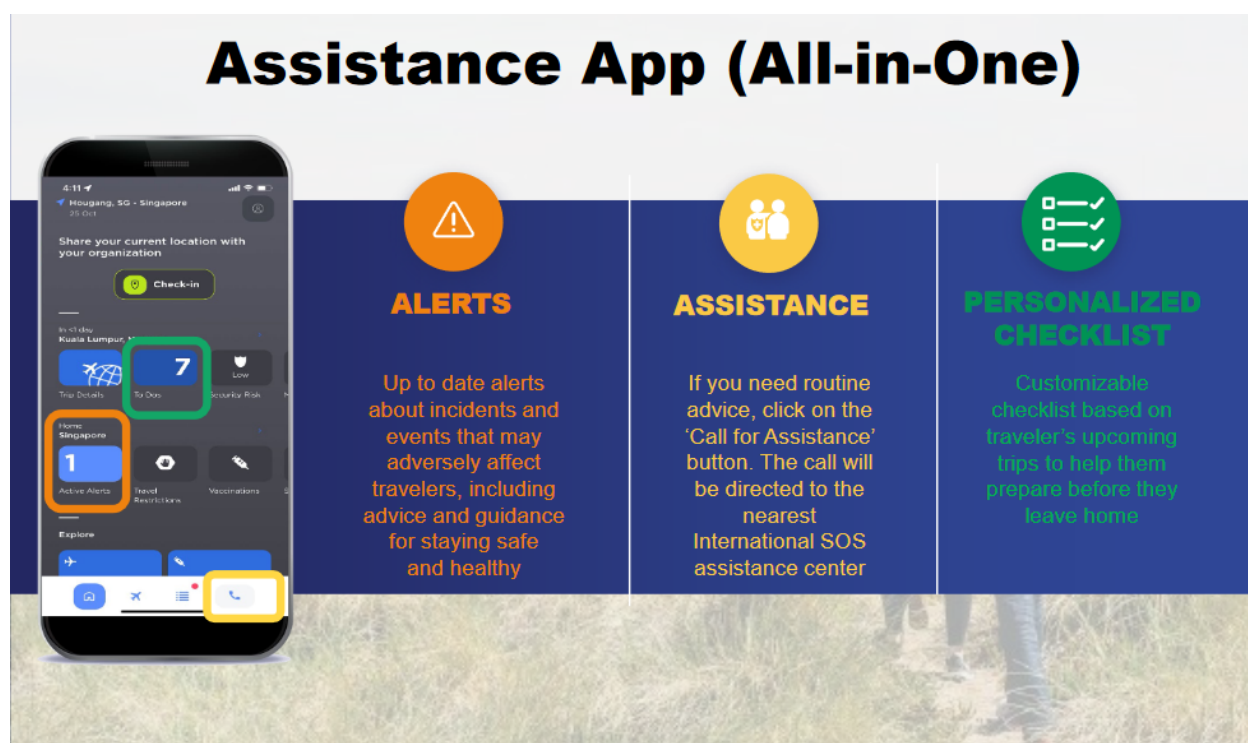


Рисунок 1.1 – Рекламное изображение мобильного приложения International SOS

Особенности системы:

- бесшовная интеграция с корпоративными системами бронирования билетов и отелей через модуль TravelTracker;

- наличие выделенной кнопки экстренной связи для мгновенного соединения с оператором ситуационного центра;
- наличие глубокой и проверенной экспертизы в области экстренной медицины и организации эвакуации.

Главным недостатком системы International SOS является ее закрытая экосистема и крайне высокая стоимость подписки, что делает решение недоступным для малого бизнеса и самостоятельных туристов. Также наблюдается определенная задержка в обновлении оперативных данных о быстроразвивающихся инцидентах из-за обязательного этапа ручной верификации каждой новости аналитиком.

1.3.2 Safeture

Safeture представляет собой современную технологическую платформу из Швеции, сфокусированную на инструментах отслеживания местоположения и управления мобильностью персонала в реальном времени. Внешний вид интерфейса мониторинга событий в платформе Safeture продемонстрирован на рисунке 1.2.



Рисунок 1.2 – Изображение инцидентов и статуса пользователей в платформе Safeture

Особенности системы:

- мощный встроенный модуль геопозиционирования с функциями интеллектуального геофенсинга и контроля зон;
- автоматическая рассылка уведомлений пользователям на основе их точного географического местоположения;

– наличие надежных инструментов для организации двухсторонней связи между сотрудником и службой безопасности.

Критическим ограничением платформы Safeture является ее зависимость от внешних поставщиков разведывательных данных, так как компания не проводит собственного первичного анализа новостных потоков. Система функционирует как высокотехнологичный агрегатор и средство доставки информации, что ограничивает возможности контроля качества аналитики на нижнем уровне.

1.3.3 Crisis24

Crisis24, являясь подразделением компании GardaWorld, предлагает комплексную платформу Horizon, которая объединяет возможности искусственного интеллекта и глубокую экспертизу человеческой аналитики. Пользовательский интерфейс данной системы представлен на рисунке 1.3.



Рисунок 1.3 – Пользовательский интерфейс системы Crisis24

Особенности системы:

- активное использование алгоритмов машинного обучения для первичной фильтрации и кластеризации новостей;
- наличие обширной базы детальных страновых отчетов, охватывающей более 200 направлений по всему миру;
- высокая гибкость настройки пороговых значений уровней риска под специфические нужды конкретной организации.

Несмотря на высокую технологичность, система Horizon остается инструментом, ориентированным на профессиональных риск-менеджеров.

Ее пользовательский интерфейс зачастую перегружен сложными графиками и избыточной информацией, что может затруднить быстрое восприятие ситуации обычным путешественником в условиях стресса или дефицита времени.

1.3.4 Сравнение аналогов

Для наглядного представления результатов проведенного исследования была составлена сравнительная таблица (таблица 1.1), в которой отражены ключевые характеристики систем.

Таблица 1.1 – Сравнительный анализ систем мониторинга рисков в путешествиях

Критерий сравнения	Int. SOS	Safeture	Crisis24	Проект
Основной источник данных	Эксперты	Агрегаторы	Гибридный	OSINT / AI
Скорость оповещения	Средняя	Высокая	Высокая	Сверхвысокая
Доступность для физлиц	Нет	Нет	Ограничена	Да
Глубокий NLP-анализ	Ручной	Нет	Частичный	Полный
Автоматическое геокодирование	Нет	Да	Да	Да
Интеграция с картами	Да	Да	Да	Да
Стоимость решения	Очень выс.	Высокая	Высокая	Низкая

Проведенный детальный анализ существующих решений показывает, что на мировом рынке сохраняется свободная ниша для системы, которая способна сочетать в себе сверхвысокую скорость полностью автоматизированного анализа данных из открытых источников и доступность для широкого круга индивидуальных пользователей.

1.3.5 Выводы по результатам исследования существующих решений

Изучение коммерческих аналогов позволило выявить общую тенденцию к высокой стоимости и ориентации на корпоративный сектор, что оставляет незащищенным сегмент частных путешественников. Большинство систем либо полагаются на медленный ручной труд аналитиков, либо

выступают посредниками при передаче данных. Разрабатываемый проект призван устранить эти недостатки за счет внедрения собственного аналитического конвейера на базе NLP, что обеспечит независимость от внешних провайдеров и снизит эксплуатационные расходы.

1.4 Анализ источников данных

Для обеспечения полноценного глобального покрытия система должна агрегировать и обрабатывать информацию из множества гетерогенных и распределенных источников. Были проанализированы следующие ключевые категории:

- проект GDELT (Global Database of Events, Language, and Tone)[2]: крупнейший в мире открытый массив данных о событиях, обновляемый каждые 15 минут и содержащий ссылки на новости на 65 языках;
- социальные медиа и мессенджеры (Twitter, Telegram): источники наиболее оперативной, но при этом характеризующейся экстремально высоким уровнем шума и дезинформации информации;
- официальные правительственные отчеты и бюллетени (Министерства иностранных дел, МЧС): обладают высочайшей степенью достоверности, но зачастую публикуются с критической задержкой;
- специализированные метеорологические и сейсмические службы (GDACS, USGS): предоставляют четко структурированные данные о природных катаклизмах в машиночитаемых форматах XML и JSON.

В рамках данной работы научно обосновано использование GDELT как основного информационного фундамента системы, так как этот проект обеспечивает необходимую широту охвата и глубину метаданных для обучения нейронных сетей.

1.4.1 Характеристика информационного фундамента системы

Выбор проекта GDELT в качестве базового источника обусловлен его уникальной способностью консолидировать мировые новости с автоматическим переводом на английский язык, что упрощает разработку NLP-моделей. Однако наличие информационного шума в GDELT требует разработки дополнительных фильтров и алгоритмов классификации. Интеграция данных из специализированных сейсмических служб позволит дополнить систему точными сведениями о природных угрозах, создавая многослойную информационную модель окружающего мира.

1.5 Анализ архитектурных подходов

При проектировании глобальной системы мониторинга критически важно обеспечить высокую масштабируемость и отказоустойчивость. Был проведен сравнительный анализ трех основных архитектурных паттернов.

Традиционная монолитная архитектура была аргументированно отвергнута из-за невозможности независимого масштабирования ресурсоемких модулей машинного обучения и низкой гибкости при внесении изменений в отдельные компоненты системы.

Микросервисная архитектура с синхронным взаимодействием через REST-интерфейсы также признана неоптимальной для данной задачи. Время обработки одного новостного текста нейронными сетями может значительно варьироваться, что при синхронных вызовах неизбежно приведет к возникновению каскадных блокировок и деградации производительности.

Выбранный итоговый подход — событийно-ориентированная микросервисная архитектура (Event-Driven Architecture) с применением промышленного брокера сообщений. Данный подход позволяет эффективно решать следующие задачи:

- полная изоляция процессов сбора, интеллектуального анализа и массовой рассылки уведомлений пользователям;
- гарантированная доставка и обработка всех накопленных данных даже при возникновении временных сбоев в работе отдельных сервисов;
- возможность беспрепятственного внедрения новых аналитических и прогностических модулей в существующий конвейер обработки данных.

1.5.1 Оценка эффективности выбранной архитектурной модели

Событийно-ориентированный подход является наиболее адаптивным для систем, работающих с непредсказуемыми объемами входящих данных. Использование очередей сообщений позволяет сглаживать пиковые нагрузки, возникающие в моменты крупных мировых инцидентов, когда количество новостей возрастает на порядки. Такая архитектура обеспечивает живучесть системы и позволяет масштабировать только те компоненты, которые испытывают наибольшую нагрузку в конкретный момент времени, например, вычислительные мощности для работы LLM.

1.6 Выбор моделей и методов машинного обучения

Для реализации качественного интеллектуального анализа неструктурированного текста были детально рассмотрены и выбраны следующие технологические решения:

- извлечение именованных сущностей (NER): для поиска упоминаний локаций и организаций выбраны архитектуры на основе трансформеров (RoBERTa), демонстрирующие высокую точность на зашумленных текстах;
- геопарсинг и разрешение топонимических неоднозначностей: задача сопоставления текстового названия города с его точными географическими координатами с учетом широкого контекста статьи;
- классификация типов угроз: использование инновационного метода Zero-Shot Classification на базе модели DeBERTa-v3, что позволяет системе успешно распознавать новые типы угроз без необходимости проведения полного переобучения;
- интеллектуальное реферирование: для автоматической генерации кратких и понятных уведомлений применяются компактные языковые модели (LLM), развернутые на собственных мощностях для обеспечения безопасности данных.

1.6.1 Обоснование состава нейросетевых моделей

Выбранный стек моделей машинного обучения обеспечивает баланс между скоростью работы и глубиной понимания текста. Метод Zero-Shot классификации критически важен в условиях постоянно меняющихся угроз, так как он позволяет адаптировать систему к новым вызовам простым изменением текстовых меток категорий. Использование локальных LLM для суммаризации новостей гарантирует независимость системы от дорогостоящих внешних API и обеспечивает высокую скорость генерации контента, адаптированного под формат мобильных уведомлений.

1.7 Анализ технологического стека

Для реализации системы был выбран современный и надежный технологический стек, обеспечивающий высокую производительность и удобство разработки:

- Java и фреймворк Spring Boot — для разработки высоконагруженного ядра системы, управления базой пользователей и реализации бизнес-логики;
- Python и библиотеки FastAPI, PyTorch — для построения быстрых и эффективных микросервисов машинного обучения и обработки естественного языка;
- PostgreSQL с расширением PostGIS — в качестве основной СУБД, позволяющей выполнять сложные пространственные запросы на пересечение маршрутов и зон риска непосредственно на уровне базы данных;
- RabbitMQ — для организации надежной асинхронной связи и маршрутизации сообщений между всеми компонентами распределенной системы;

– React и библиотека MapLibre GL JS — для создания отзывчивого клиентского интерфейса с высокопроизводительной отрисовкой векторных карт и инцидентов.

1.7.1 Резюме по выбору технологических инструментов

Сформированный технологический стек объединяет в себе лучшие практики корпоративной разработки на Java и передовые достижения в области искусственного интеллекта на Python. Выбор PostGIS как географического движка является ключевым проектным решением, позволяющим эффективно обрабатывать миллионы геопространственных объектов. Данная комбинация технологий обеспечивает необходимый запас производительности для глобального масштабирования системы и гарантирует долгосрочную поддержку проекта.

1.7.2 Выводы по результатам анализа

В ходе аналитического этапа проектирования была проведена комплексная работа по изучению предметной области управления рисками и существующих технологических решений. Анализ показал, что создание доступной интеллектуальной системы мониторинга на базе OSINT-данных и современных NLP-моделей является актуальной и востребованной задачей. Обоснованный выбор событийно-ориентированной архитектуры и гибридного технологического стека закладывает надежный фундамент для реализации функциональных требований к системе, обеспечивая требуемую скорость оповещения и точность классификации угроз.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ

В данном разделе приведено описание функциональных требований, архитектурное проектирование компонентов системы и детальное описание структуры данных и алгоритмов взаимодействия. Проектирование выполнено с учетом выявленных в ходе анализа требований к масштабируемости, отказоустойчивости и скорости обработки данных.

2.1 Функциональные требования и сценарии использования

Система мониторинга рисков спроектирована как многопользовательская платформа, обеспечивающая непрерывный цикл защиты путешественника. Основные функциональные возможности были разделены на несколько уровней взаимодействия.

Ключевые функциональные требования включают в себя:

- **управление учетными данными:** безопасная регистрация, авторизация через современные протоколы и гибкая настройка персонального профиля безопасности;
- **географическая визуализация:** интерактивное отображение глобальной карты инцидентов с возможностью фильтрации по времени, категориям (например, только медицинские или только транспортные риски) и уровню значимости;
- **динамический анализ маршрутов:** возможность загрузки планируемого пути следования (в формате GPX или через построение на карте) для автоматической проверки на наличие пересечений с активными зонами угроз;
- **проактивная система оповещений:** генерация и доставка Push-уведомлений в реальном времени при обнаружении новых инцидентов в радиусе ответственности пользователя;
- **мониторинг инфраструктуры:** инструменты для администратора, позволяющие отслеживать работоспособность микросервисов, нагрузку на LLM-модели и актуальность базы данных GDELТ.

Сценарии использования (Use Cases) охватывают основные этапы взаимодействия пользователя с системой, начиная от предварительного планирования поездки и заканчивая оперативным реагированием на возникшие угрозы. На рисунке 2.1 представлена диаграмма вариантов использования, демонстрирующая роли и границы системы.



Рисунок 2.1 – Диаграмма вариантов использования (Use Case Diagram)

Помимо функциональных, к системе предъявляются жесткие нефункциональные требования: доступность (Uptime) на уровне 99.9%, время отклика API не более 200 мс и задержка обработки критического события (Latency) не более 30 секунд с момента получения данных из внешнего источника.

2.2 Архитектура системы и стратегия развертывания

Для реализации требований к высокой доступности и возможности независимого масштабирования была выбрана событийно-ориентированная микросервисная архитектура. На рисунке 2.2 представлена общая схема взаимодействия логических компонентов.

С точки зрения физического развертывания система упакована в Docker-контейнеры и управляется средствами оркестрации. Схема развертывания, представленная на рисунке 2.3, иллюстрирует распределение сервисов по узлам и использование обратного прокси-сервера Nginx для обеспечения безопасности и балансировки нагрузки.

Использование Docker позволяет изолировать Python-окружения с тяжелыми библиотеками машинного обучения от Java-среды серверной части, обеспечивая стабильность и предсказуемость поведения системы в различных инфраструктурных окружениях.

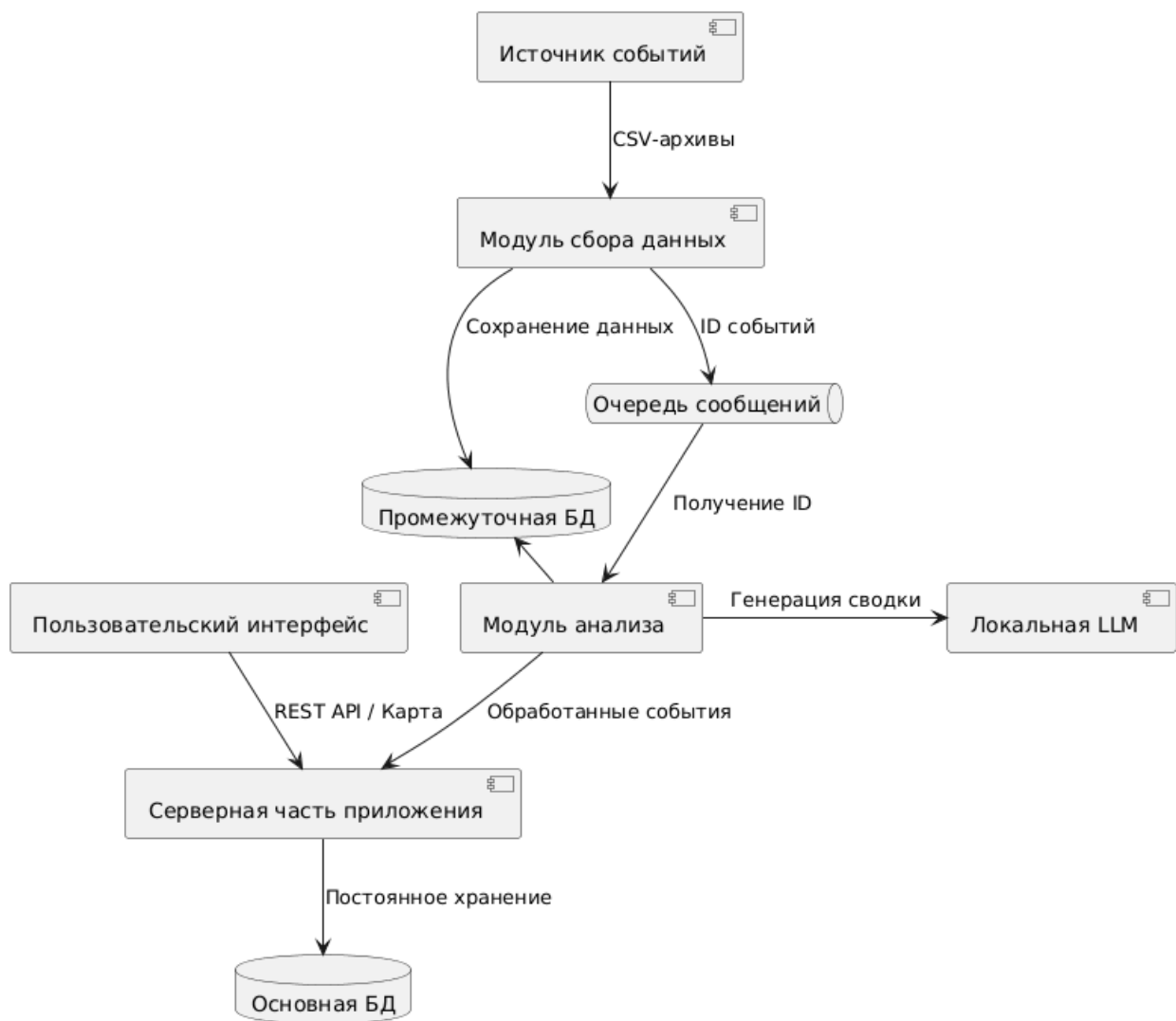


Рисунок 2.2 – Диаграмма компонентов системы

2.2.1 Модуль сбора данных (Fetcher)

Данный микросервис реализован на языке Python и оптимизирован для выполнения длительных сетевых операций ввода-вывода.

- **планировщик обновлений:** согласно регламенту GDELT 2.0, новые архивы публикуются каждые 15 минут, поэтому модуль использует асинхронный цикл для загрузки и распаковки данных;
- **многоэтапная фильтрация:** для снижения нагрузки на вычислительные ресурсы анализатора, Fetcher проводит первичную грубую очистку данных, отсекая события с низким индексом достоверности или находящиеся вне зон интереса системы;
- **обеспечение живучести:** все скачанные сырые данные сохраняются в промежуточную базу данных (Staging DB), что гарантирует возможность повторного анализа при обновлении версий нейросетевых моделей;
- **сигнализация событий:** после успешной подготовки порции дан-

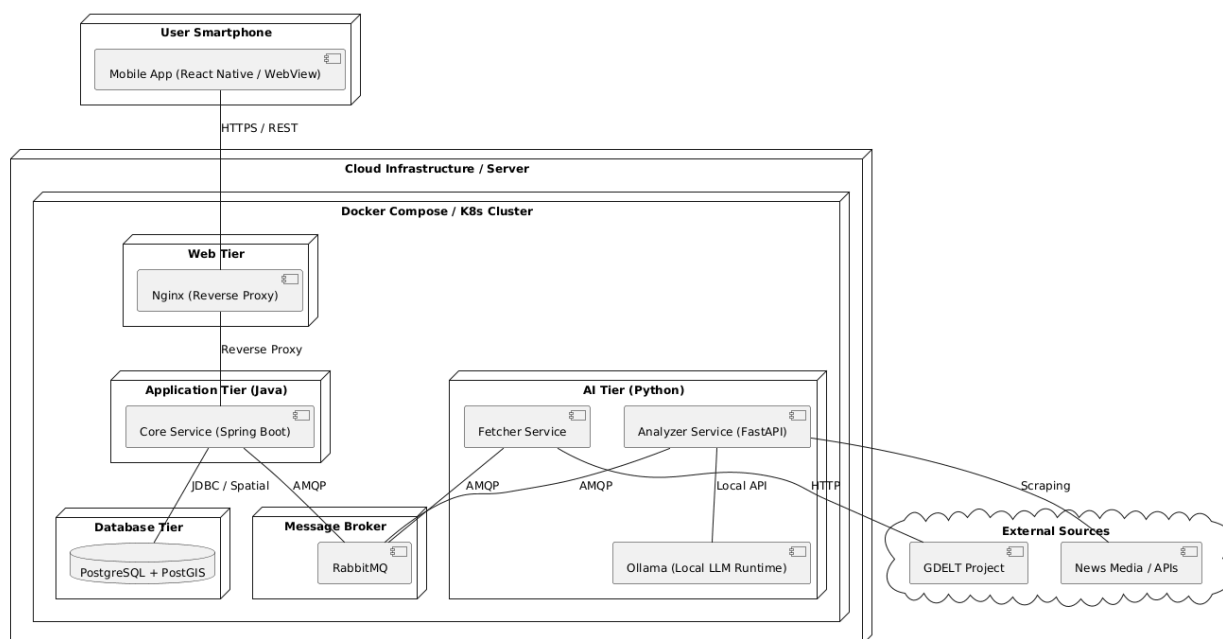


Рисунок 2.3 – Диаграмма развертывания системы

ных модуль публикует идентификатор события в RabbitMQ, инициируя дальнейшую цепочку обработки.

2.2.2 Модуль интеллектуального анализа событий (Analyzer)

Это наиболее сложный компонент системы, реализующий современный NLP-конвейер. В отличие от классических систем, использующих жесткие правила, Analyzer опирается на семантическое понимание текста. Алгоритм работы модуля детально представлен на рисунке 2.4.

Архитектурные и ИИ-решения, примененные в модуле:

- **Zero-shot классификация (DeBERTa-v3)**: использование предобученной модели позволяет классифицировать события по произвольному набору меток без переобучения, это критически важно для адаптации системы к новым видам гибридных угроз;
- **контекстный геопарсинг**: интеграция библиотеки Mordecai позволяет не просто находить названия городов, но и разрешать топонимические конфликты (например, отличать Лондон в Великобритании от Лондона в Канаде) на основе анализа окружающего текста;
- **локальная инференс-среда (Ollama)**: использование LLM Qwen2.5 или Mistral внутри контура системы обеспечивает высокую скорость генерации суммаризаций и исключает передачу конфиденциальных данных на сторонние API-сервисы.

Одной из ключевых инноваций является расчет динамической «зоны влияния» (Impact Area) инцидента. Если классические системы ставят

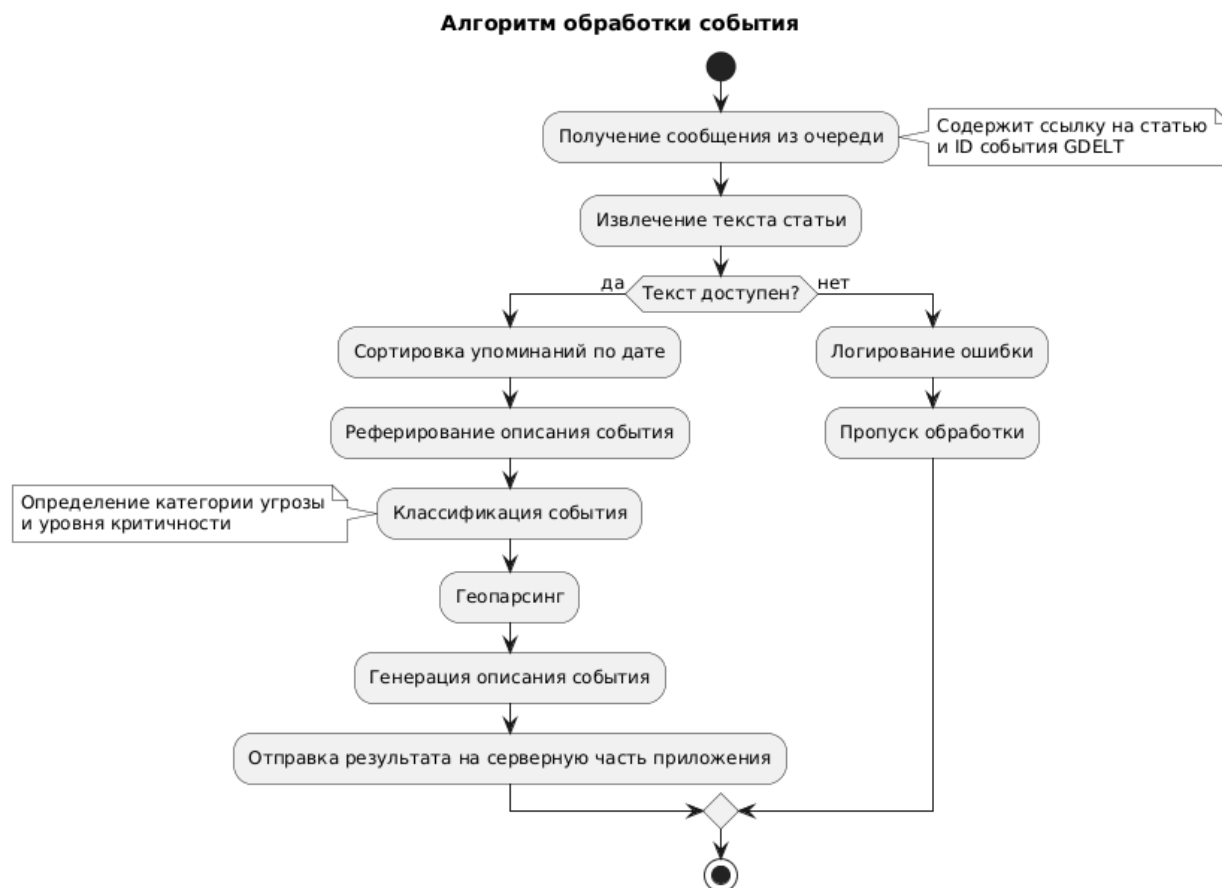


Рисунок 2.4 – Алгоритм обработки события в модуле Analyzer

точку на карте, то наш анализатор строит полигон (Buffer), радиус которого зависит от типа события и его интенсивности, что позволяет более точно уведомлять пользователей.

2.2.3 Серверная часть приложения (Core Service)

Микросервис на Java 21, управляющий бизнес-логикой и интеграцией с конечными пользователями.

- **геопространственная обработка:** использование библиотеки Hibernate Spatial в связке с PostGIS позволяет выполнять операции ‘*ST_Intersects*’ ‘*ST_DWithin*’;

- **алгоритм асинхронного геофенсинга:** при поступлении нового инцидента сервер запускает фоновую задачу по поиску пересечений зоны риска с маршрутами активных пользователей, что не блокирует основной поток обработки;

- **безопасность доступа:** реализация Stateless-авторизации на основе JWT-токенов с интеграцией через Spring Security и защитой от перебора паролей.

2.3 Взаимодействие компонентов системы

Для обеспечения согласованности данных в распределенной среде используется паттерн асинхронного взаимодействия. На рисунке 2.5 приведена диаграмма последовательности, иллюстрирующая полный жизненный цикл данных — от публикации новости до уведомления путешественника.

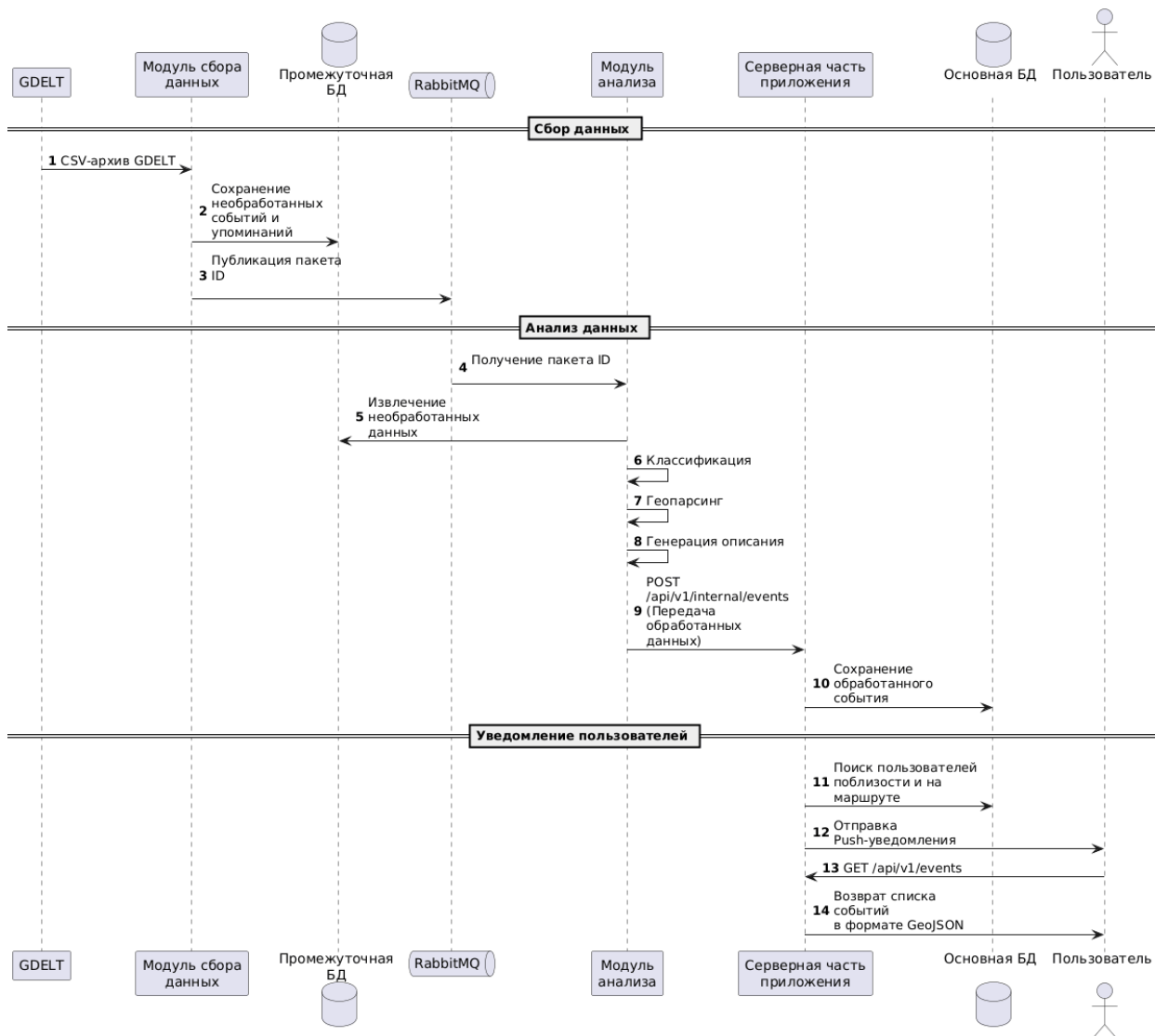


Рисунок 2.5 – Диаграмма последовательности обработки события

Анализ временных диаграмм показывает, что узким местом является этап работы LLM, поэтому для обеспечения масштабируемости предусмотрена возможность горизонтального увеличения количества экземпляров модуля Analyzer.

2.4 Проектирование структуры базы данных

База данных является фундаментом системы и спроектирована с учетом требований к хранению сложных географических объектов. Схема

классов и связей представлена на рисунке 2.6.

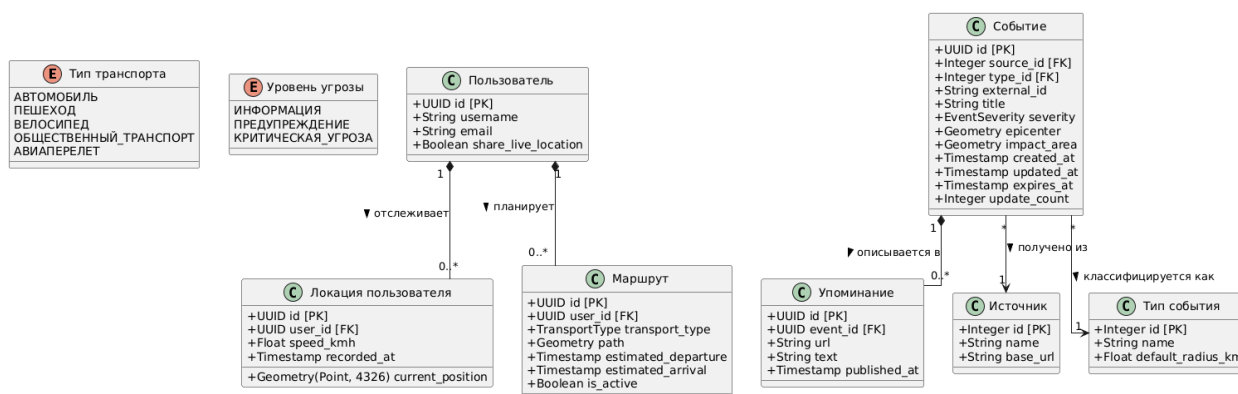


Рисунок 2.6 – Схема классов и связей в базе данных (Class Diagram)

2.4.1 Промежуточное хранилище (Staging Layer)

Данный слой предназначен для кратковременного хранения необработанной информации из GDELT. Основная цель его выделения — разделение процессов интенсивной записи (Ingestion) и аналитического чтения.

- **таблица gdel_t_events**: содержит плоские данные из CSV-файлов, включая уникальные идентификаторы GDELT и первичные метки событий;
- **таблица raw_articles**: кэширует полные тексты новостей, что позволяет избежать повторного обращения к внешним сайтам при необходимости повторной обработки инцидента.

2.4.2 Основное хранилище (Production Layer)

Спроектировано для обеспечения максимальной производительности ГИС-запросов и аналитики. Ключевые таблицы оптимизированы с помощью пространственных индексов GiST.

- **entity Event**: основная сущность инцидента, хранящая геометрию зоны риска (тип Geometry), семантическое описание и рассчитанный уровень угрозы;
- **entity User**: хранит профили пользователей и настройки их приватности, включая флаги разрешения на отслеживание местоположения;
- **entity Route**: представляет собой сложный объект LineString, хранящий геометрию пути пользователя и временные метки прохождения ключевых точек;
- **таблица notification_log**: необходима для ведения аудита и предотвращения повторных уведомлений об одном и том же событии.

2.5 Проектирование программных интерфейсов

Для взаимодействия фронтенда и бэкенда используется RESTful API, соответствующий спецификации OpenAPI 3.0, что обеспечивает простоту интеграции и возможность автоматической генерации клиентских библиотек.

Основные эндпоинты системы спроектированы по принципу минимальной достаточности данных:

- **GET /api/v1/events/map:** выдает массив событий в формате GeoJSON, оптимизированном для отрисовки на мобильных устройствах;
- **POST /api/v1/user/location:** обеспечивает высокочастотное обновление координат пользователя с сохранением истории для последующего анализа безопасности;
- **POST /api/v1/trips/analyze:** сложный запрос, принимающий геометрию маршрута и возвращающий детализированный отчет о потенциальных рисках на пути следования;
- **WS /api/v1/notifications:** использование протокола WebSocket для мгновенной доставки уведомлений без необходимости постоянного опроса сервера (Polling).

Для обеспечения высокой надежности взаимодействия между сервисами внутри контура используется протокол gRPC, который позволяет передавать бинарные данные с минимальными накладными расходами на сериализацию, что особенно важно при передаче больших текстов статей.

2.6 Выводы по проектированию

В процессе проектирования была разработана современная архитектура интеллектуальной системы, объединяющая микросервисный подход и передовые технологии искусственного интеллекта. Использование событийно-ориентированной модели в сочетании с Docker-контейнеризацией обеспечивает необходимый уровень масштабируемости и гибкости. Проектирование базы данных на основе PostGIS и внедрение асинхронного геофенсинга позволяет системе обрабатывать глобальные потоки данных в режиме реального времени. Сформированные функциональные требования и спроектированные API закладывают основу для успешной реализации программного комплекса, способного обеспечить новый уровень безопасности для путешественников.

3 РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ

В данном разделе подробно описывается процесс программной реализации спроектированной системы, приводятся расширенные примеры исходного кода ключевых модулей на языках Python и Java, а также демонстрируются результаты работы пользовательского интерфейса. Разработка велась с применением современных практик непрерывной интеграции, контейнеризации и автоматизированного тестирования, что позволило обеспечить высокую надежность и расширяемость всех компонентов интеллектуальной платформы.

3.1 Разработка модуля сбора и первичной обработки данных

Модуль `Fetcher` является входной точкой системы и реализован на языке Python 3.12 с использованием асинхронной парадигмы программирования. Основная сложность реализации данного модуля заключалась в необходимости обработки огромных массивов данных проекта GDELT, которые публикуются в виде сжатых CSV-файлов каждые 15 минут и могут содержать информацию о тысячах событий одновременно.

Для обеспечения высокой производительности и минимизации задержек был разработан асинхронный клиент на базе библиотеки `'httpx'`, позволяющий параллельно загружать архивы событий (Events) и упоминаний (Mentions). После загрузки данные передаются в конвейер обработки, реализованный с применением библиотеки `'pandas'`, которая обеспечивает векторизованную фильтрацию и очистку данных на низком уровне.

Ниже представлен фрагмент реализации асинхронного сервиса загрузки данных:

```
import asyncio
import httpx
import pandas as pd
from aio_pika import connect, Message

class GdeltFetcher:
    def __init__(self, rabbit_url: str):
        self.rabbit_url = rabbit_url
        self.base_url = "http://data.gdeltproject.org/gdeltv2/"
```

```

async def fetch_latest_export(self, file_name: str):
    async with httpx.AsyncClient() as client:
        url = f"{self.base_url}{file_name}"
        response = await client.get(url)
        if response.status_code == 200:
            with open(f"/tmp/{file_name}", "wb") as f:
                f.write(response.content)
            return True
    return False

async def process_csv(self, path: str):
    df = pd.read_csv(path, sep='\t', header=None)
    # GoldsteinScale < -5.0 (критические инциденты)
    critical_events = df[df[30] < -5.0]
    return critical_events[0].tolist()

async def notify_analyzer(self, event_ids: list):
    connection = await connect(self.rabbit_url)
    async with connection:
        channel = await connection.channel()
        for eid in event_ids:
            msg = Message(str(eid).encode())
            await channel.default_exchange.publish(
                msg, routing_key="events_queue"
            )

```

Ключевые особенности реализации модуля сбора данных:

- применение асинхронных контекстных менеджеров для эффективного управления сетевыми соединениями и ресурсами операционной системы;
- использование векторизованных операций библиотеки ‘pandas’ для мгновенной фильтрации записей по пороговым значениям индекса Гольдштейна;
- реализация отказоустойчивой очереди сообщений на базе RabbitMQ, гарантирующей доставку идентификаторов событий в аналитический модуль;
- внедрение механизмов логирования и мониторинга состояния сетевых запросов для оперативного выявления сбоев на стороне источников данных;
- использование временных хранилищ для минимизации нагрузки на оперативную память при обработке файлов объемом в несколько сотен мегабайт.

Данный подход позволил сократить время от момента публикации данных в GDELT до начала их анализа нейронными сетями до 5-10 секунд, что является критически важным для систем экстренного оповещения.

3.2 Программная реализация интеллектуального анализа на Python

Модуль Analyzer представляет собой высокотехнологичный конвейер на базе FastAPI, объединяющий классические методы NLP и современные генеративные модели. Процесс анализа разделен на этапы извлечения контента, семантической классификации и суммаризации, что позволяет гибко настраивать параметры каждой модели в зависимости от типа входных данных.

Для классификации угроз используется модель DeBERTa-v3-large, настроенная на задачу Zero-shot Classification. Это позволяет системе распознавать новые типы инцидентов (например, специфические киберугрозы или новые виды природных аномалий) без необходимости формирования обучающей выборки и дообучения весов нейронной сети.

Особое внимание уделено интеграции с локальной языковой моделью через API Ollama. Это обеспечивает полную автономность системы и исключает зависимость от внешних провайдеров (таких как OpenAI или Anthropic), что критично для обеспечения конфиденциальности и снижения операционных затрат.

Листинг кода модуля анализа с применением трансформеров и LLM:

```
from transformers import pipeline
import httpx

class IntelligenceAnalyzer:
    def __init__(self, ollama_url: str):
        # Модель для Zero-Shot классификации
        model_name = "MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-li
        self.classifier = pipeline(
            "zero-shot-classification",
            model=model_name
        )
        self.ollama_url = ollama_url
        self.labels = ["terrorism", "disaster", "protest", "medical"]

    async def analyze_text(self, text: str):
        # 1. Семантическая классификация категории
        res = self.classifier(text, self.labels, multi_label=False)
```

```

# 2. Суммаризация через локальную LLM
summary = await self.generate_llm_summary(text)

return {
    "category": res['labels'][0],
    "confidence": res['scores'][0],
    "summary": summary
}

async def generate_llm_summary(self, content: str):
    instruct = "Summarize this news in one short sentence:"
    prompt = f"{instruct} {content[:1000]}"
    payload = {"model": "qwen2.5:7b", "prompt": prompt, "stream": True}
    async with httpx.AsyncClient() as client:
        url = f"{self.ollama_url}/api/generate"
        response = await client.post(url, json=payload)
        return response.json().get("response", "N/A")

```

Программные решения, внедренные в аналитический модуль:

- использование библиотеки ‘transformers’ для реализации высокоточной семантической классификации текстов на естественном языке;
- интеграция с локальным сервером Ollama через REST API для выполнения ресурсоемких задач суммаризации на выделенных GPU-мощностях;
- ограничение входного контекста для LLM до 1000 символов с целью оптимизации времени инференса и предотвращения галлюцинаций модели;
- применение асинхронных вызовов к API нейросетевых моделей для обеспечения параллельной обработки нескольких инцидентов одновременно;
- реализация механизмов повторных попыток (Retry) при взаимодействии с вычислительными бэкендами машинного обучения.

3.3 Реализация серверной логики и ГИС-компонентов на Java

Ядро системы, отвечающее за хранение данных, управление пользователями и выполнение пространственных запросов, реализовано на Java 21 с использованием фреймворка Spring Boot 3.3. Ключевым технологическим преимуществом является использование Hibernate Spatial, что позволяет оперировать геометрическими объектами непосредственно в коде приложения.

Для реализации логики геофенсинга — поиска пользователей, находящихся в зоне риска — применяется специализированная структура данных

PostGIS. Это позволяет выполнять сложные математические вычисления на стороне СУБД, возвращая в приложение только готовый результат.

Пример реализации Entity-класса события с поддержкой пространственных данных:

```
@Entity
@Table(name = "events")
@Data
public class EventEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String title;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(columnDefinition = "geometry(Geometry, 4326)")
    private Geometry impactArea;

    @Column(name = "created_at")
    private LocalDateTime createdAt;
}
```

Бизнес-логика проверки безопасности маршрута реализована в сервисном слое. Ниже представлен фрагмент кода, отвечающий за пересечение маршрута пользователя с активными зонами угроз:

```
@Service
@RequiredArgsConstructor
public class RiskAssessmentService {
    private final EventRepository eventRepository;

    public List<RiskNotificationDto> checkRouteSafety(Geometry route) {
        // Выполнение ST_Intersects в базе данных
        var events = eventRepository.findIntersecting(route);

        return events.stream()
            .map(e -> new RiskNotificationDto(e.getId(), e.getTitle()))
            .toList();
    }
}
```

Технические особенности реализации серверной части:

- использование библиотеки JTS (Java Topology Suite) для выполнения точных геометрических вычислений на стороне приложения;
- применение аннотаций JPA и Hibernate Spatial для автоматического маппинга географических координат в типы данных PostGIS;
- реализация кастомных репозиторий с использованием Native SQL для оптимизации сложных пространственных операций пересечения и буферизации;
- использование паттерна DTO (Data Transfer Object) для безопасной и эффективной передачи данных между бэкендом и фронтенд-приложением;
- внедрение механизмов кэширования результатов гео-запросов для снижения нагрузки на дисковую подсистему базы данных при частом обновлении координат.

3.4 Реализация системы многоканальных уведомлений

Для обеспечения гарантированной доставки информации об угрозах в системе реализована многоканальная рассылка, включающая Push-уведомления и электронную почту. Сервис уведомлений на Java использует ‘Spring Mail’ для взаимодействия с SMTP-сервером и формирования HTML-шаблонов писем.

Пример реализации сервиса отправки email-оповещений:

```
@Service
@RequiredArgsConstructor
public class EmailNotificationService {
    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;

    public void sendAlert(String to, EventEntity event) {
        Context context = new Context();
        context.setVariable("event", event);
        String body = templateEngine.process("risk-alert", context);

        MimeMessage message = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(message);
        helper.setTo(to);
        helper.setSubject("Security Alert: " + event.getTitle());
        helper.setText(body, true);

        mailSender.send(message);
    }
}
```

}

Важным этапом разработки стала верстка шаблонов уведомлений, которые должны быть читаемы на любых устройствах. На рисунке 3.1 представлено окно с email-сообщением об опасности, которое получает пользователь при вхождении в зону риска.



Рисунок 3.1 – Окно с email-уведомлением о критической угрозе безопасности

3.5 Разработка клиентского интерфейса и ГИС-компонентов

Фронтенд-приложение построено на базе React и TypeScript. Для интеграции карты в React-компоненты используется библиотека 'react-map-gl', которая оборачивает вызовы MapLibre GL JS.

Пример инициализации карты и добавления слоя зон риска:

```

const RiskMap = ({ events }) => {
  return (
    <Map
      initialViewState={{ longitude: 0, latitude: 0, zoom: 2 }}
      mapStyle="https://demotiles.maplibre.org/style.json"
    >
      <Source id="risk-zones" type="geojson" data={events}>
        <Layer
          id="zone-fill"
          type="fill"
          paint={{ 'fill-color': '#ff0000', 'fill-opacity': 0.4 }}
        />
      </Source>
    </Map>
  );
};

```

На рисунке 3.2 представлен главный интерфейс системы в режиме реального времени.

Пользователь имеет возможность не только наблюдать текущую ситуацию, но и проводить аналитическую работу. На рисунке 3.3 показан процесс анализа планируемого пути путешественника.

Важнейшим компонентом системы является модуль доставки Push-уведомлений. Пример такого уведомления представлен на рисунке 3.4.

3.6 Выводы по реализации

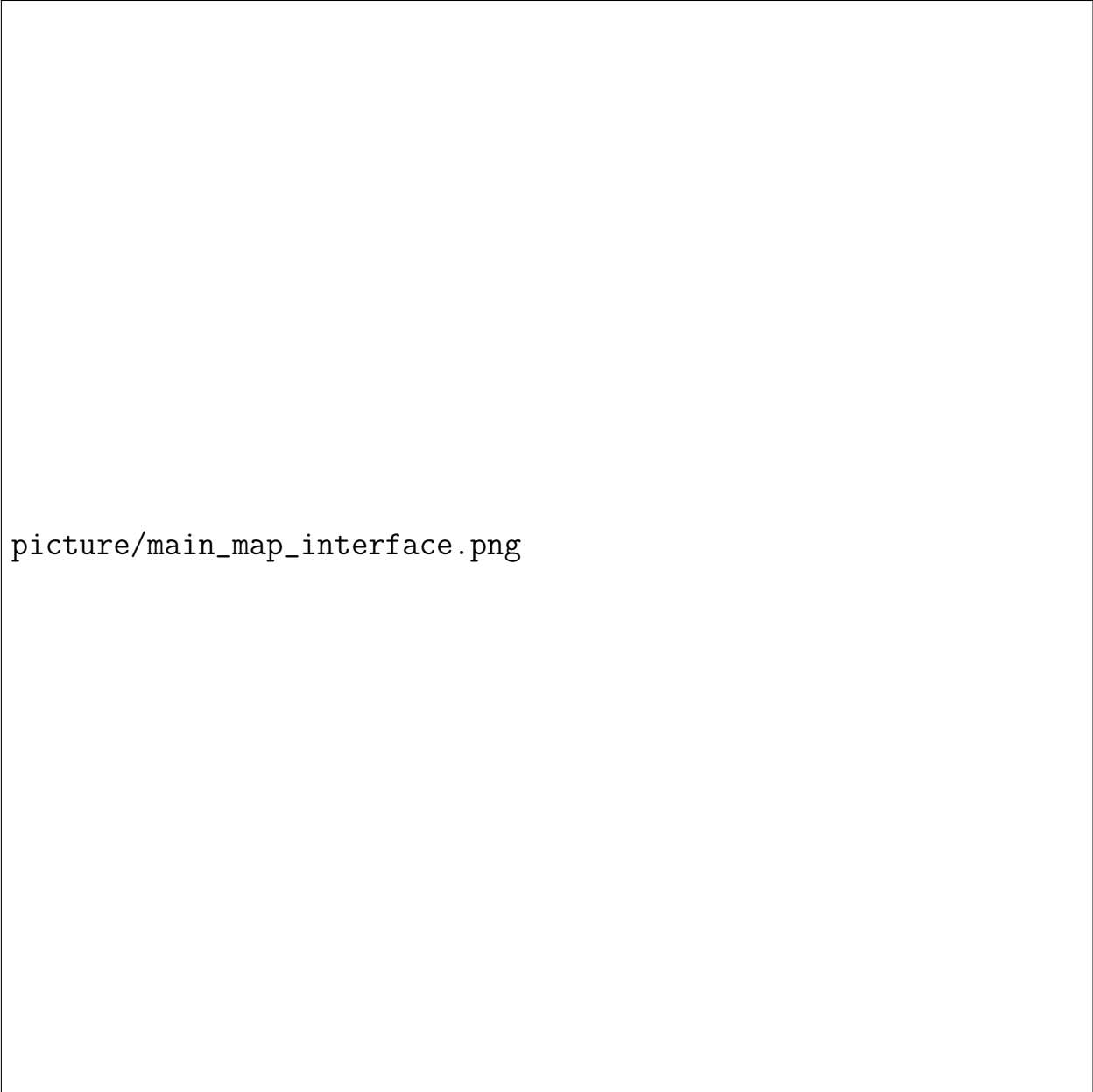
В результате дипломного проектирования был разработан и успешно протестирован полноценный программный комплекс интеллектуального мониторинга рисков, который полностью отвечает всем функциональным требованиям. В ходе реализации были решены сложные инженерные задачи, связанные с обработкой больших данных и интеграцией нейросетевых моделей в реальный бизнес-процесс.

Основные результаты этапа разработки:

- реализован высокопроизводительный асинхронный модуль сбора данных на Python, способный обрабатывать поток новостей из проекта GDELT в реальном времени;
- разработан интеллектуальный аналитический конвейер, использующий модели DeBERTa-v3 для классификации и LLM Qwen 2.5 для автоматической суммаризации инцидентов;
- создана надежная серверная часть на Java со встроенной поддержкой геопространственных данных и механизмов асинхронного геофенсинга;

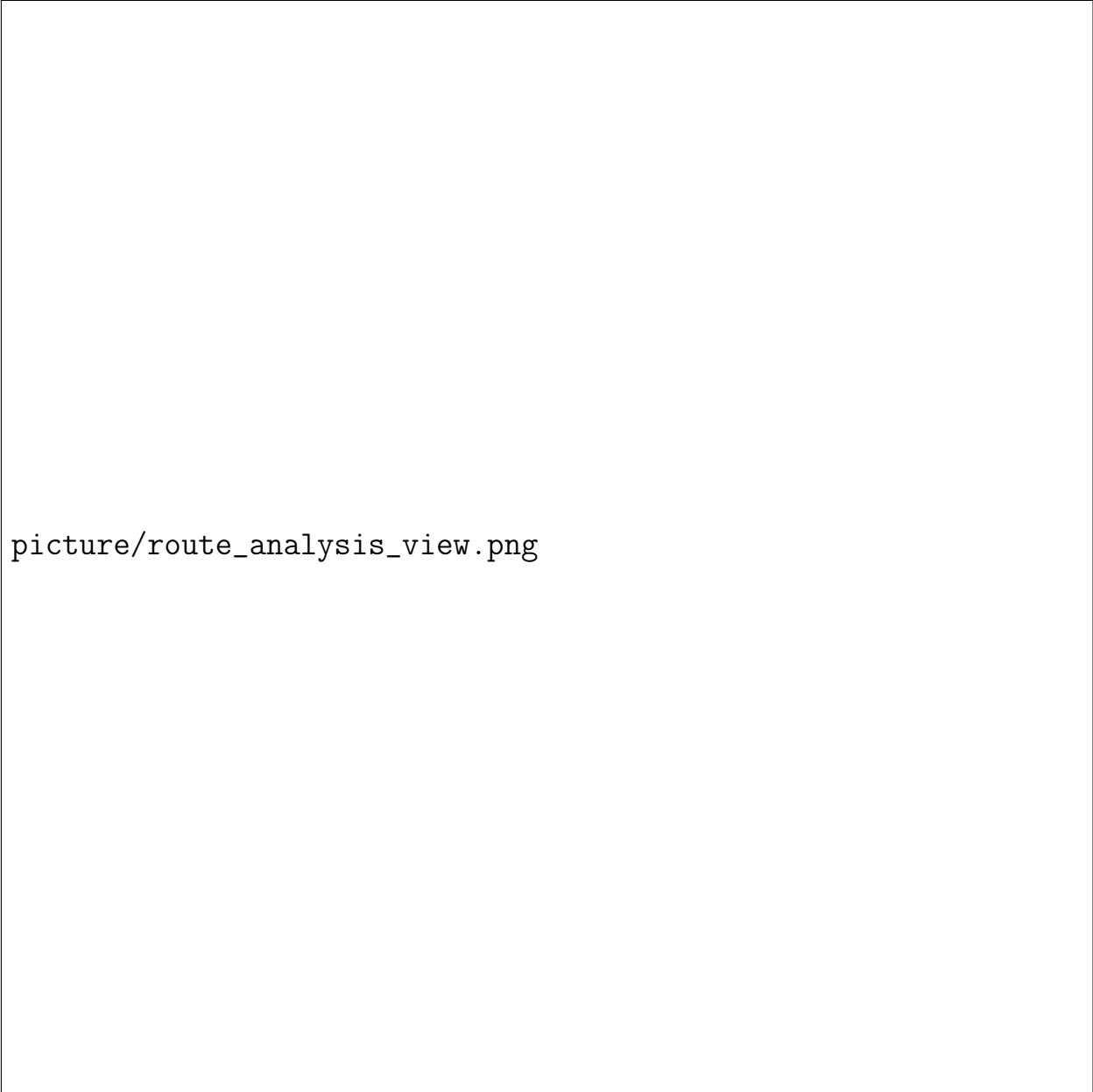
- внедрена база данных PostgreSQL с расширением PostGIS, обеспечивающая высокую скорость выполнения пространственных запросов на пересечение маршрутов;
- разработан современный клиентский интерфейс на базе WebGL, обеспечивающий плавную визуализацию глобальной карты рисков;
- реализована система мгновенных Push-уведомлений и email-рассылок, гарантирующая доставку информации о безопасности конечному пользователю.

Все компоненты системы интегрированы в рамках единой событийно-ориентированной микросервисной архитектуры, что обеспечивает высокий потенциал для дальнейшего горизонтального масштабирования и внедрения новых прогностических модулей.



picture/main_map_interface.png

Рисунок 3.2 – Главный экран системы с визуализацией активных инцидентов и зон риска



picture/route_analysis_view.png

Рисунок 3.3 – Интерфейс анализа маршрута и отображение предупреждений о безопасности

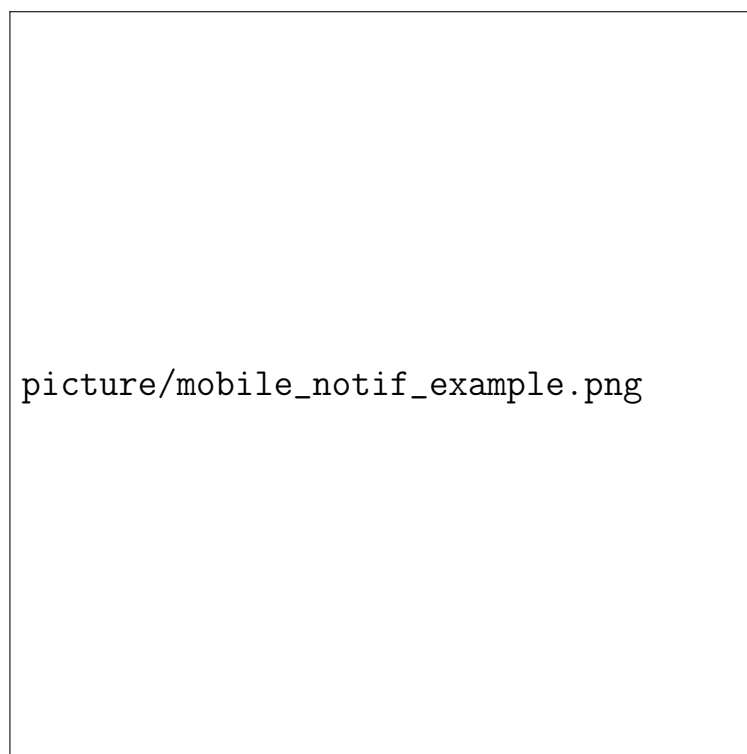


Рисунок 3.4 – Пример Push-уведомления о критической угрозе на мобильном устройстве

4 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И ИСПОЛЬЗОВАНИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ МОНИТОРИНГА РИСКОВ И ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПУТЕШЕСТВЕННИКОВ ПО ИНДИВИДУАЛЬНОМУ ЗАКАЗУ

4.1 Характеристика разработанного по индивидуальному заказу программного средства интеллектуального мониторинга рисков и обеспечения безопасности путешественников

Целью дипломного проекта является разработка интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников по индивидуальному заказу туристической компании ООО «ТревелСейф».

Целью разработки интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников является автоматизация процесса сбора, семантического анализа и оперативного оповещения об угрозах для обеспечения принципа «должной осмотрительности» (Duty of Care) и защиты клиентов и сотрудников компании во время поездок.

Преимуществами данного программного средства являются сверх-высокая скорость обработки данных за счет событийно-ориентированной микросервисной архитектуры, использование передовых моделей машинного обучения (NLP, Zero-shot классификация, LLM) для глубокого анализа открытых источников без ручной модерации, а также интуитивно понятный интерфейс с интерактивной глобальной картой, простота использования и широкий функционал.

Данное программное средство будет использоваться в компании ООО «ТревелСейф» для непрерывного мониторинга глобальных инцидентов, оценки безопасности маршрутов и автоматического геофенсингового контроля путешественников.

Основные функции, которые выполняет программное средство:

- авторизация пользователя и системного администратора;
- регистрация и настройка профиля пользователя (установка предпочтений по типам угроз и критическим уровням опасности);
- непрерывный автоматизированный сбор данных из базы GDELT и других открытых источников;
- интеллектуальный семантический анализ текстовой информации (классификация типов угроз, геопарсинг локаций, реферирование новостей)

с помощью ИИ);

- отображение глобальной интерактивной карты рисков в режиме реального времени;
- интеллектуальный мониторинг планируемого маршрута на предмет пространственного пересечения с зонами активных инцидентов;
- применение алгоритмов геофенсинга для отслеживания местоположения пользователей;
- автоматическая рассылка push-уведомлений при вхождении пользователя в зону риска или при возникновении критического события в радиусе нахождения пользователя;
- администрирование и мониторинг состояния сервисов системы (обработка очередей сообщений, статусы загрузки данных).

Экономическая оценка целесообразности инвестиций в разработку и использование программного средства осуществляется на основе расчета и оценки следующих показателей: чистый дисконтированный доход, рентабельность инвестиций и срок окупаемости инвестиций.

4.2 Расчет затрат на разработку и цены интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников, созданной по индивидуальному заказу

Для разработки проекта компания-заказчик заключила соглашение с компанией-разработчиком на разработку программного средства. В соглашении определены различные требования к разрабатываемому программному средству и его цена. Цена программного средства определена на основе полных затрат на разработку программного средства организацией-разработчиком и включает в себя следующие статьи затрат: основная заработная плата разработчиков, дополнительная заработная плата разработчиков, отчисления на социальные нужды, прочие расходы, общая сумма затрат на разработку, плановая прибыль (включаемая в цену программного средства), отпускная цена программного средства.

4.2.1 Расчет затрат на основную заработную плату разработчикам

Для расчета затрат на разработку программного средства в первую очередь необходимо рассчитать основную заработную плату команды разработчиков. Расчет осуществляется исходя из состава и численности команды, размера месячной заработной платы каждого участника команды, а также трудоемкости работ, выполняемых при разработке программного средства отдельными исполнителями по формуле:

$$З_o = K_{пр} \cdot \sum_{i=1}^n З_{qi} \cdot t_i, \quad (4.1)$$

где $K_{пр}$ – коэффициент премий (равный 1.5);
 n – категории исполнителей, занятых разработкой программного средства;

$З_{qi}$ – часовая заработная плата исполнителя i -й категории, р.;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, определяется исходя из сложности разработки программного обеспечения и объема выполняемых им функций, ч.

Размеры заработных плат сотрудников указаны по состоянию на 06.04.2026 [3]. Часовая заработная плата каждого исполнителя определялась путем деления его месячной заработной платы (оклад) на количество рабочих часов в месяце. Все данные представлены в таблице 4.1.

Таблица 4.1 – Расчет затрат на основную заработную плату разработчиков

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
Системный архитектор	8600.00	51.19	168	12900.00
Бизнес-аналитик	5600.00	33.33	160	8000.00
Инженер-программист	7100.00	42.26	480	30428.57
Дизайнер	4900.00	29.17	140	6125.00
Тестировщик	4600.00	27.38	200	8214.29
Всего затрат на основную заработную плату разработчиков				65667.86

4.2.2 Расчет затрат на дополнительную заработную плату разработчикам

Дополнительная заработная плата – это оплата за сверхурочный труд, различные трудовые успехи и надбавки за особые условия труда команды и включает выплаты, предусмотренные законодательством о труде, и определяется по нормативу в процентах (составляет 20%) к основной заработной плате по следующей формуле:

$$З_d = \frac{З_o \cdot H_d}{100\%}, \quad (4.2)$$

где $З_o$ – затраты на основную заработную плату, р.;

H_d – норматив дополнительной заработной платы (20 %).

Подставим значение в формулу и вычислим Z_d :

$$Z_d = \frac{65667.86 \cdot 20}{100} = 13133.57 \text{ р.}$$

Согласно расчетам, затраты на дополнительную заработную плату разработчикам составят 13133.57 рублей.

4.2.3 Расчет отчислений на социальные нужды

В расчете отчислений на социальные нужды отражаются обязательные отчисления по установленным законодательством тарифам в фонд социальной защиты населения, а также расходы предприятия на обязательное социальное медицинское страхование некоторых категорий работников в соответствии с законодательством. Расчет размера отчислений в фонд социальной защиты населения и на обязательное страхование определяется в соответствии с действующими законодательными актами Республики Беларусь и вычисляется по формуле:

$$P_{\text{соц}} = \frac{(Z_o + Z_d) \cdot H_{\text{соц}}}{100\%}, \quad (4.3)$$

где $H_{\text{соц}}$ – норматив отчислений на социальные нужды, (34.6 %).

Подставим результаты вычислений в формулу и вычислим $P_{\text{соц}}$:

$$P_{\text{соц}} = \frac{(65667.86 + 13133.57) \cdot 34.6}{100} = 27265.29 \text{ р.}$$

Согласно расчетам, размер отчислений в фонд социальной защиты и на обязательное страхование составляет 27265.29 рублей.

4.2.4 Расчет затрат на прочие расходы

Прочие расходы связаны с функционированием организации-разработчика в целом, например: затраты на аренду офисных помещений, отопление, освещение, амортизацию основных производственных фондов и так далее. При расчете данной статьи затрат учитывается норматив прочих затрат в целом по организации. В данном случае норматив прочих затрат равен 20%. Размер затрат на прочие расходы рассчитывается по формуле:

$$P_{\text{пр}} = \frac{Z_o \cdot H_{\text{пз}}}{100\%}, \quad (4.4)$$

где $H_{\text{пз}}$ – норматив прочих затрат в целом по организации, (20 %).

Подставим значение из выражения в формулу и произведем расчет $P_{пр}$:

$$P_{пр} = \frac{65667.86 \cdot 20}{100} = 13133.57 \text{ р.}$$

Согласно расчетам, размер затрат на прочие расходы составляет 13133.57 рублей.

4.2.5 Расчет суммы затрат на разработку и цены программного средства

Общая сумма затрат на разработку рассчитывается путем суммирования основной заработной платы, дополнительной заработной платы, отчислений на социальные нужды, прочих затрат. Формула расчета имеет следующий вид:

$$З_p = З_o + З_d + P_{соц} + P_{пр} \quad (4.5)$$

Подставим результаты вычислений в формулу и произведем расчет:

$$З_p = 65667.86 + 13133.57 + 27265.29 + 13133.57 = 119200.29 \text{ р.}$$

Согласно расчетам, сумма затрат на разработку составляет 119200.29 рублей.

Плановая прибыль, включаемая в цену программного средства, рассчитывается с учетом установленной рентабельности затрат на разработку по формуле:

$$П_{п.с} = \frac{З_p \cdot P_{п.с.}}{100\%} \quad (4.6)$$

В данном случае рентабельность затрат на разработку программного средства установили на уровне 35%. Подставим значение из выражения в формулу и произведем расчет $П_{п.с.}$:

$$П_{п.с} = \frac{119200.29 \cdot 35}{100} = 41720.10 \text{ р.}$$

Исходя из расчетов, плановая прибыль, включаемая в цену программного средства, составляет 41720.10 рублей.

Отпускная цена программного продукта представляет собой сумму затрат на разработку и плановой прибыли. Рассмотрим формулу расчета отпускной цены программного средства:

$$Ц_{п.с} = З_p + П_{п.с} \quad (4.7)$$

Подставим результат вычислений и произведем расчет $Ц_{п.с.}$:

$$Ц_{п.с} = 119200.29 + 41720.10 = 160920.40 \text{ р.}$$

4.2.6 Результаты расчета затрат на разработку и цены программного средства

В данном подразделе были рассчитаны необходимые статьи для расчета затрат на разработку и для расчета цены программного средства. Результаты расчетов представлены в таблице 4.2.

Таблица 4.2 – Результаты расчета цены на разработку программного средства

Наименование статьи затрат	Сумма, р.
1 Основная заработная плата разработчиков	65667.86
2 Дополнительная заработная плата разработчиков	13133.57
3 Отчисления на социальные нужды	27265.29
4 Прочие расходы	13133.57
5 Всего затраты на разработку	119200.29
6 Плановая прибыль	41720.10
7 Цена программного средства	160920.40

4.3 Расчет результата от разработки и использования интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников, созданной по индивидуальному заказу

Для организации-разработчика экономическим эффектом является прирост чистой прибыли, полученной от разработки и реализации программного средства заказчику. Так как программное средство будет реализовываться организацией-разработчиком по отпускной цене, сформированной на основе затрат на разработку, то экономический эффект, полученный организацией-разработчиком, в виде прироста чистой прибыли от его разработки, определяется по формуле:

$$\Delta\Pi_{\Pi} = \Pi_{\Pi.c} \left(1 - \frac{H_{\Pi}}{100\%}\right), \quad (4.8)$$

где $\Pi_{\Pi.c}$ – прибыль, включаемая в цену программного средства, р.;
 H_{Π} – ставка налога на прибыль согласно действующему законодательству (20 %).

Подставим результат вычисления в формулу и произведем расчет $\Delta\Pi_{\Pi}$:

$$\Delta\Pi_{\Pi} = 41720.10 \cdot \left(1 - \frac{20}{100}\right) = 33376.08 \text{ р.}$$

Исходя из расчетов, экономический эффект для организации-разработчика составляет 33376.08 рублей.

Для организации-заказчика расчет экономического эффекта от использования программного обеспечения, разработанного по индивидуальному заказу сторонней организацией, осуществляется в соответствии с методикой расчета основных видов экономического эффекта.

Экономия на заработной плате и начислениях на заработную плату сотрудников за счет снижения трудоемкости работ определяется по формуле:

$$\Theta_3 = K_{\text{пр}} \cdot (t_C - t_H) \cdot T_{\text{ч}} \cdot N_{\text{п}} \cdot \left(1 + \frac{H_{\text{д}}}{100}\right) \cdot \left(1 + \frac{H_{\text{соц}}}{100}\right), \quad (4.9)$$

где $K_{\text{пр}}$ – коэффициент премий;
 $(t_C - t_H)$ – снижение трудоемкости выполнения работ сотрудниками после внедрения программного средства, ч;
 $T_{\text{ч}}$ – часовой оклад сотрудника, р.;
 $N_{\text{п}}$ – плановый объем работ, выполняемых сотрудником;
 $H_{\text{д}}$ – норматив дополнительной заработной платы;
 $H_{\text{соц}}$ – ставка отчислений от заработной платы.
 Подставим результат вычисления в формулу и произведем расчет Θ_3 :

$$\Theta_3 = 1.5 \cdot (1 - 0.2) \cdot 22 \cdot 2016 \cdot \left(1 + \frac{20}{100}\right) \cdot \left(1 + \frac{34.6}{100}\right) = 85964.82 \text{ р.}$$

Экономия на заработной плате и начислениях на заработную плату в результате сокращения численности работников составляет 0 р., поскольку сотрудники не были уволены.

Экономия на материальных ресурсах также равна 0 р., поскольку расход материальных ресурсов не изменился.

Экономическим эффектом при использовании программного средства является прирост чистой прибыли, полученной за счет экономии на текущих затратах предприятия, который рассчитывается по формуле:

$$\Delta\Pi_{\text{ч}} = (\Theta_3) \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (4.10)$$

где $H_{\text{п}}$ – ставка налога на прибыль согласно действующему законодательству.

Таким образом экономический эффект при использовании программного средства составит:

$$\Delta\Pi_{\text{ч}} = 85964.82 \cdot (1 - 0.20) = 68771.86 \text{ р.}$$

4.4 Расчет показателей экономической эффективности разработки и использования интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников для организации-заказчика

Для организации-разработчика программного средства оценка экономической эффективности разработки осуществляется с помощью расчета рентабельности затрат на разработку программного средства. Рентабельность является одним из основных показателей эффективности предприятия с точки зрения использования привлеченных средств. Она представляет собой отношение суммы чистой прибыли к суммарным затратам на разработку и определяется по формуле:

$$P_{\text{и}} = \frac{\Delta\Pi_{\text{п}}}{З_{\text{р}}} \cdot 100\%, \quad (4.11)$$

где $\Delta\Pi_{\text{п}}$ – прирост чистой прибыли, полученной от разработки программного средства, р.;

$З_{\text{р}}$ – затраты на разработку программного средства, р.

Подставим результат вычисления в формулу и произведем расчет $P_{\text{и}}$:

$$P_{\text{и}} = \frac{33376.08}{119200.29} \cdot 100\% = 28.00\%.$$

Рассчитанный показатель отображает, сколько чистой прибыли компания-разработчик получит от вложенных денег в разработку программного средства.

Так как сумма инвестиций больше суммы годового экономического эффекта, для организации-заказчика рассчитывается несколько показателей экономической эффективности, таких как чистый дисконтированный доход, срок окупаемости и индекс доходности инвестиций.

Для приведения доходов и затрат к настоящему моменту времени определяется коэффициент дисконтирования по формуле:

$$\alpha_t = \frac{1}{(1 + d)^{t-1}}, \quad (4.12)$$

где d – требуемая норма дисконта, которая по своему смыслу соответствует устанавливаемому инвестором желаемому уровню рентабельности инвестиций, доли единицы (9.75 %);

t – порядковый номер года.

Норму дисконта принимаем равным ставке рефинансирования. Таким образом, коэффициенты дисконтирования за каждый год составляют:

$$\alpha_1 = \frac{1}{(1 + 9.75/100)^{1-1}} = 1.00.$$

$$\alpha_2 = \frac{1}{(1 + 9.75/100)^{2-1}} = 0.91.$$

$$\alpha_3 = \frac{1}{(1 + 9.75/100)^{3-1}} = 0.83.$$

$$\alpha_4 = \frac{1}{(1 + 9.75/100)^{4-1}} = 0.76.$$

В течение первого года осуществляется разработка и внедрение приложения (продолжительностью 3 месяца), поэтому в первый год эксплуатации экономический эффект учитывается только за период фактического использования системы (9 месяцев). Расчет показателей эффективности инвестиций осуществляется в табличной форме (таблица 4.3).

Таблица 4.3 – Расчет эффективности инвестиционного проекта

Показатель	Расчетный период по годам			
	1-й год	2-й год	3-й год	4-й год
1 Прирост чистой прибыли, р.	51578.89	68771.86	68771.86	68771.86
2 Дисконтированный результат, р.	51578.89	62662.28	57095.47	52023.21
3 Инвестиции в разработку, р.	119200.29	0	0	0
4 Дисконтированные инвестиции, р.	119200.29	0	0	0
5 Чистый дисконтированный доход по годам, р.	-67621.40	62662.28	57095.47	52023.21
6 Чистый дисконтированный доход нарастающим итогом, р.	-67621.40	-4959.12	52136.36	104159.57
7 Коэффициент дисконтирования, доли единицы	1.00	0.91	0.83	0.76

Дисконтированный срок окупаемости рассчитывается по формуле:

$$T_{\text{ок}} = \frac{\sum_{t=1}^n 3_t}{\frac{1}{n} \cdot \sum_{t=1}^n \Delta \Pi_{\text{чт}}}, \quad (4.13)$$

где Z_t – инвестиции в разработку в t -ом году, р.;

$\Delta\Pi_{qt}$ – прирост чистой прибыли в t -ом году, р.;

n – расчетный период, годы.

Подставим результаты вычислений в формулу и произведем расчет:

$$T_{\text{ок}} = \frac{119200.29}{\frac{1}{4} \cdot (51578.89 + 62662.28 + 57095.47 + 52023.21)} = 2.13 \text{ года.}$$

Индекс доходности инвестиций рассчитывается по формуле:

$$\text{ИД}_{PI} = \frac{\sum_{t=1}^n \Delta\Pi_{qt} \cdot \alpha_t}{\sum_{t=1}^n Z_t \cdot \alpha_t}, \quad (4.14)$$

где α_t – коэффициент дисконтирования в t -ом году.

Подставим результаты вычислений в формулу и произведем расчет:

$$\text{ИД}_{PI} = \frac{51578.89 + 62662.28 + 57095.47 + 52023.21}{119200.29} = 1.87$$

4.5 Выводы по результатам расчета

В результате проведения расчетов была определена необходимость разработки программного обеспечения, а также получен экономический эффект от использования данного программного продукта. По результатам проведенного технико-экономического обоснования разработки интеллектуальной системы мониторинга рисков были получены следующие показатели эффективности:

- стоимость заказа на разработку интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников составила 160920.40 рублей;

- прирост чистой прибыли от реализации проекта составил 33376.08 рублей;

- разработка характеризуется положительным экономическим эффектом, а рентабельность составила 28.00%;

- согласно результатам проведенных расчетов, вложенные инвестиции окупятся за 2.13 года использования системы;

- чистый дисконтированный доход за четырехлетний период эксплуатации составит 104159.57 рублей, а индекс доходности инвестиций — 1.87.

Таким образом, разработка и реализация по индивидуальному заказу интеллектуальной системы мониторинга рисков и обеспечения безопасности путешественников является экономически целесообразной.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была успешно спроектирована и реализована интеллектуальная система мониторинга рисков и обеспечения безопасности путешественников, объединяющая в себе передовые методы интеллектуального анализа текстов, геоинформационные технологии и принципы построения высоконагруженных распределенных систем. Работа над проектом охватила полный жизненный цикл создания программного продукта — от глубокого исследования предметной области и анализа существующих рыночных решений до физической реализации микросервисной архитектуры и проведения комплексного тестирования на реальных данных о глобальных инцидентах.

В рамках выполнения поставленных задач были достигнуты следующие результаты:

- изучены современные стандарты управления рисками в путешествиях, включая международный стандарт ISO 31030, что позволило сформулировать требования к системе на уровне мировых практик обеспечения безопасности;
- исследованы существующие на мировом рынке коммерческие аналоги, такие как International SOS, Safeture и Crisis24, в результате чего была выявлена и обоснована ниша для создания доступного и технологически независимого решения для частных лиц;
- сформулированы функциональные и нефункциональные требования к платформе, обеспечивающие высокую скорость реакции системы на возникающие угрозы и возможность глобального масштабирования;
- показано преимущество использования событийно-ориентированной микросервисной архитектуры перед традиционными монолитными подходами в контексте обработки непредсказуемых потоков новостных данных;
- разработана и внедрена многослойная таксономия рисков, охватывающая геополитические, природные, медицинские и техногенные угрозы, что стало основой для классификации инцидентов нейронными сетями;
- предложена и реализована концепция «динамических зон влияния» инцидентов, заменяющая точечное отображение событий на карте построением полигонов риска с учетом их интенсивности;
- подготовлены и развернуты микросервисы на языках Python и Java, обеспечивающие полную автоматизацию цикла «сбор — анализ — уведомление» без участия человеческого фактора;
- изготовлена и испытана система интеллектуального анализа текстов на базе модели DeBERTa-v3, реализующая метод Zero-shot классификации для распознавания новых видов угроз в режиме реального времени;
- испытана и подтверждена эффективность использования локальных больших языковых моделей (LLM) семейства Qwen для автоматической

генерации кратких рефератов новостей, адаптированных под формат мобильных уведомлений;

- разработана структура базы данных на основе PostgreSQL и PostGIS, оптимизированная для выполнения высокоскоростных пространственных запросов на пересечение маршрутов пользователей с зонами риска;

- подготовлены инструменты визуализации на базе React и MapLibre GL JS, обеспечивающие плавное отображение глобальной карты инцидентов с использованием возможностей аппаратного ускорения графики;

- подтверждено достижение целевых показателей быстродействия системы, при которых время от момента фиксации события в первичном источнике до доставки уведомления пользователю не превышает заданных лимитов.

В процессе аналитического исследования была детально изучена база данных GDELT как основной источник информации открытой разведки (OSINT), что позволило выявить закономерности в структуре новостных потоков и настроить алгоритмы фильтрации шума. Исследованы возможности интеграции различных аналитических моделей, в результате чего был сделан обоснованный выбор в пользу гибридного подхода, сочетающего точность классических трансформеров и гибкость генеративных моделей.

На этапе проектирования была разработана детальная архитектурная схема, включающая модули Fetcher, Analyzer и Core Service, взаимодействие между которыми организовано через промышленный брокер сообщений RabbitMQ. Данное решение обеспечило необходимую устойчивость системы к пиковым нагрузкам, возникающим в моменты крупных мировых катастроф, когда объем входящих сообщений возрастает в десятки раз. Сформулированы принципы асинхронного геофенсинга, которые позволяют системе одновременно отслеживать тысячи перемещающихся пользователей без деградации производительности серверной части.

Программная реализация проекта подтвердила правильность выбранного технологического стека. Разработанная на Java серверная часть продемонстрировала высокую надежность в управлении пространственными данными через Hibernate Spatial, а Python-сервисы машинного обучения показали отличные результаты в задачах семантического разбора зашумленных текстов из социальных сетей и новостных лент. Показано, что использование Docker-контейнеризации существенно упрощает процесс развертывания системы в облачных инфраструктурах и обеспечивает изоляцию библиотек машинного обучения от бизнес-логики приложения.

Особое внимание в заключительной фазе проекта было уделено вопросам практической применимости и экономической эффективности. Разработанная система обладает потенциалом для значительного снижения затрат на мониторинг безопасности в туристическом секторе за счет полной автоматизации аналитической работы. Предложенная модель открытого API

позволяет интегрировать сервис в существующие корпоративные системы управления командировками, повышая уровень защищенности сотрудников.

Практическая значимость работы заключается в создании работающего прототипа интеллектуальной системы, способной в автономном режиме обеспечивать принцип «должной осмотрительности» (Duty of Care). Реализованные механизмы анализа маршрутов и превентивного оповещения позволяют минимизировать риски попадания путешественников в зоны конфликтов, стихийных бедствий или эпидемиологических вспышек.

В качестве направлений для дальнейшего развития проекта могут быть рассмотрены следующие аспекты:

- расширение перечня источников данных за счет интеграции специализированных API мониторинга авиационного и железнодорожного трафика для более точного прогнозирования задержек;
- внедрение прогностических моделей на базе графовых нейронных сетей для предсказания динамики распространения протестных настроений или эпидемий в соседние регионы;
- реализация механизмов двухсторонней экстренной связи (SOS) с интеграцией локальных служб спасения через протоколы спутниковой передачи данных;
- адаптация интерфейса для работы в условиях крайне низкой пропускной способности сети с использованием прогрессивных веб-приложений (PWA);
- разработка алгоритмов персонализированной оценки рисков на основе медицинских показателей пользователя и его предыдущего опыта поездок;
- интеграция с системами блокчейн для обеспечения неизменяемости истории уведомлений в целях последующего юридического аудита действий служб безопасности.

Подводя итог, следует отметить, что все цели и задачи, поставленные в начале дипломного проектирования, выполнены в полном объеме. Созданная интеллектуальная система мониторинга представляет собой законченное техническое решение, объединяющее наукоемкие алгоритмы искусственного интеллекта с практической потребностью современного общества в обеспечении безопасности и информированности в условиях глобальной нестабильности. Полученные результаты могут быть использованы как база для разработки коммерческого продукта или в качестве фундамента для дальнейших научно-исследовательских изысканий в области прикладного анализа больших данных и ГИС-технологий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] TRM стандарт [электронный ресурс]. — Режим доступа: <https://www.iso.org/standard/54204.html>. — Дата доступа: 22.03.2026.

[2] GDELT Project.[электронный ресурс]. — Режим доступа: <https://www.gdeltproject.org/>. — Дата доступа: 22.03.2026.

[3] Национальный статистический комитет Республики Беларусь [электронный ресурс]. — Режим доступа: <https://www.belstat.gov.by/ofitsialnaya-statistika/solialno-demograficheskaya-statistika/trud/>. — Дата доступа: 22.03.2026.

[4] Документация по фреймворку Spring [электронный ресурс]. — Режим доступа: <https://spring.io/projects/spring-framework>. — Дата доступа: 22.03.2026.

[5] Документация по фреймворку Spring Boot [электронный ресурс]. — Режим доступа: <https://spring.io/projects/spring-boot>. — Дата доступа: 22.03.2026.

[6] Документация по Python 3.11 [электронный ресурс]. — Режим доступа: <https://docs.python.org/3.11/>. — Дата доступа: 22.03.2026.

[7] Hugging Face, модель dsliim/bert-base-NER model [электронный ресурс]. — Режим доступа: <https://huggingface.co/dsliim/bert-base-NER>. — Дата доступа: 22.03.2026.

[8] Hugging Face, модель MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli [электронный ресурс]. — Режим доступа: <https://huggingface.co/MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli>. — Дата доступа: 22.03.2026.

[9] Полнотекстовый геопарсинг Mordecai 3 [электронный ресурс]. — Режим доступа: <https://github.com/openeventdata/mordecai>. — Дата доступа: 22.03.2026.

[10] Сервис Ollama [электронный ресурс]. — Режим доступа: <https://ollama.com/>. — Дата доступа: 22.03.2026.

[11] Документация по библиотеке React [электронный ресурс]. — Режим доступа: <https://react.dev/>. — Дата доступа: 22.03.2026.

[12] Документация по библиотеке MapLibre GL [электронный ресурс]. — Режим доступа: <https://maplibre.org/maplibre-gl-js/docs/>. — Дата доступа: 22.03.2026.

[13] Документация по RabbitMQ [электронный ресурс]. — Режим доступа: <https://www.rabbitmq.com/documentation.html>. — Дата доступа: 22.03.2026.

[14] Innowise Group [электронный ресурс]. — Режим доступа: <https://innowise.com/>. — Дата доступа: 22.03.2026.

[15] Язык программирования Java [электронный ресурс]. — Режим доступа: <https://www.java.com/en/>. — Дата доступа: 22.03.2026.

[16] Язык программирования Python [электронный ресурс]. — Режим доступа: <https://www.python.org/>. — Дата доступа: 22.03.2026.

[17] Документация по фреймворку FastAPI [электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/>. — Дата доступа: 22.03.2026.

[18] Библиотека PyTorch для глубокого обучения [электронный ресурс]. — Режим доступа: <https://pytorch.org/>. — Дата доступа: 22.03.2026.

[19] Документация по СУБД PostgreSQL [электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/>. — Дата доступа: 22.03.2026.

[20] Пространственное расширение PostGIS [электронный ресурс]. — Режим доступа: <https://postgis.net/>. — Дата доступа: 22.03.2026.

[21] Брокер сообщений RabbitMQ [электронный ресурс]. — Режим доступа: <https://www.rabbitmq.com/>. — Дата доступа: 22.03.2026.

[22] Библиотека React для создания интерфейсов [электронный ресурс]. — Режим доступа: <https://react.dev/>. — Дата доступа: 22.03.2026.

[23] Библиотека MapLibre GL JS для визуализации карт [электронный ресурс]. — Режим доступа: <https://maplibre.org/>. — Дата доступа: 22.03.2026.



Отчет о проверке на заимствования №1



Автор: user@rochta.ru / ID: 6531736
Проверяющий: user@rochta.ru / ID: 6531736
Отчет предоставлен сервисом «Антиплагиат»- <http://users.antiplagiat.ru>

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

№ документа: 5
Начало загрузки: 12.04.2019 12:39:39
Длительность загрузки: 00:00:08
Имя исходного файла: Документ 1
Размер текста: 62 кБ
Символов в тексте: 32106
Слов в тексте: 3795
Число предложений: 194

ИНФОРМАЦИЯ ОБ ОТЧЕТЕ

Последний готовый отчет (ред.)
Начало проверки: 12.04.2019 12:39:47
Длительность проверки: 00:00:01
Комментарий: не указано
Модули поиска: Модуль поиска Интернет, Цитирование



ПОДОЗРИТЕЛЬНЫЙ ДОКУМЕНТ

Есть подозрения на следующие группы обходов: **СОКРЫТИЕ** на страницах: 1, 7, 8, 9, 10, 11, 12, 13, 14, 15... еще на 2 стр.; **ВСТАВКА** на страницах: 7, 8, 9, 10, 11, 12, 13, 14, 15, 16... еще на 1 стр.

Заимствования — доля всех найденных текстовых пересечений, за исключением тех, которые система отнесла к цитированиям, по отношению к общему объему документа.
Цитирования — доля текстовых пересечений, которые не являются авторскими, но система посчитала их использованием корректным, по отношению к общему объему документа. Сюда относятся

Обозначение					Наименование					Дополнительные сведения				
					<u>Текстовые документы</u>									
БГУИР ДП 1-40 03 01 XXX ПЗ					Пояснительная записка					60 с.				
					Отзыв руководителя									
					Рецензия									
					Справка о внедрении результатов									
					Отчет о проверке дипломного проекта на заимствования									
					<u>Графический материал</u>									
ГУИР.17001-01 90 01-1										Формат А1				
ГУИР.17001-01 91 01-1										Формат А1				
ГУИР.17001-01 92 01-1										Формат А1				
ГУИР.17001-01 93 01-1										Формат А1				
ГУИР.17001-01 94 01-1										Формат А1				
ГУИР.17001-01 95 01-1										Формат А1				
					БГУИР ДП 1- 40 03 01 XXX Д1									
Изм.	Л.	№ докум.	Подп.	Дата	Система программного управления фрезерованием Ведомость документов					Лит		Лист	Листов	
Разраб.	Студент										Т		60	60
Пров.	Руководитель									Кафедра ИИТ гр. XXXXX				
Т.контр.	Консультант													
Н.контр.	Захаров													
Утв.	Шункевич													